Chapter 13
Matrix Inversion

It is amusing to remark that we were so involved with
matrix inversion that we probably talked of nothing else for months.
Just in this period Mrs. von Neumann acquired a big,
rather wild but gentle Irish Setter puppy,
which she called inverse in honor of our work!

— HERMAN H. GOLDSTINE, *The Computer.'*
From Pascal to von Neumann (1972)

The most computationally intensive portion
of the tasks assigned to the processors is
integrating the KKR matrix inverse over the first Brillouin zone.
To evaluate the integral,
hundreds or possibly thousands of complex double precision matrices
of order between 80 and 300 must be formed and inverted.
Each matrix corresponds to a different vertex of the tetrahedrons
into which the Brillouin zone has been subdivided.

— M. T. HEATH, G. A. GEIST, and J. B. DRAKE, *Superconductivity*
in Early Experience with the Intel iPSC/860
at Oak Ridge National Laboratory (1990)

Press $\boxed{1/x}$ *to invert the matrix.*
Note that matrix inversion can produce erroneous results
if you are using iii-conditioned matrices.

— HEWLETT-PACKARD, *HP 48G Series User's Guide (1993)*

Almost anything you can do with A^{-1} *can be done without* it.

— GEORGE E. FORSYTHE and CLEVE B. MOLER,
Computer Solution of Linear Algebraic Systems (1967)

13.1. Use and Abuse of the Matrix Inverse

To most numerical analysts, matrix inversion is a sin. Forsythe, Malcolm, and Moler put it well when they say [395, 1977, p. 31] "In the vast majority of practical computational problems, it is unnecessary and inadvisable to actually compute A^{-1}." The best example of a problem in which the matrix inverse should not be computed is the linear equations problem $Ax = b$. Computing the solution as $x = A^{-1} \times b$ requires $2n^3$ flops, assuming A^{-1} is computed by Gaussian elimination with partial pivoting (GEPP), whereas GEPP applied directly to the system costs only $2n^3/3$ flops.

Not only is the inversion approach three times more expensive, but it is much less stable. Suppose $X = A^{-1}$ is formed *exactly*, and that the only rounding errors are in forming $x = fl(Xb)$. Then $\widehat{x} = (X + \Delta X)b$, where $|\Delta X| \leq \gamma_n |X|$, by (3.10). So $A\widehat{x} = A(X + \Delta X)b = (I + A\Delta X)b$, and the best possible residual bound is

$$|b - A\widehat{x}| \leq \gamma_n |A||A^{-1}||b|.$$

For GEPP, Theorem 9.4 yields

$$|b - A\widehat{x}| \leq 2\gamma_n |\widehat{L}||\widehat{U}||\widehat{x}|.$$

Since it is usually true that $\| \, |\widehat{L}||\widehat{U}| \, \|_\infty \approx \|A\|_\infty$ for GEPP, we see that the matrix inversion approach is likely to give a much larger residual than GEPP if A is ill conditioned and if $\|x\|_\infty \ll \| \, |A^{-1}||b| \, \|_\infty$. For example, we solved 50 25×25 systems $Ax = b$ in MATLAB, where the elements of x are taken from the normal $N(0, 1)$ distribution and A is random with $\kappa_2(A) = u^{-1/2} \approx 9 \times 10^7$. As Table 13.1 shows, the inversion approach provided much larger backward errors than GEPP in this experiment.

Given the inexpedience of matrix inversion, why devote a chapter to it? The answer is twofold. First, there *are* situations in which a matrix inverse must be computed. Examples are in statistics [54, 1974, §7.5], [721, 1984, §2.3], [744, 1989, p. 342 ff], where the inverse can convey important statistical information, in certain matrix iterations arising in eigenvalue-related problems [37, 1993], [174, 1987], [566, 1994], and in numerical integrations arising in

Table 13.1. *Backward errors* $\eta_{A,b}(\widehat{x})$ *for the* ∞*-norm.*

	min	max
$x = A^{-1} \times b$	6.66e-12	1.69e-10
GEPP	3.44e-18	7.56e-17

superconductivity computations [509, 1990] (see the quotation at the start of the chapter). Second, methods for matrix inversion display a wide variety of stability properties, making for instructive and challenging error analysis. (Indeed, the first major rounding error analysis to be published, that of von Neumann and Goldstine, was for matrix inversion-see §9.6).

Matrix inversion can be done in many different ways—in fact, there are more computationally distinct possibilities than for any other basic matrix computation. For example, in triangular matrix inversion different loop orderings are possible and either triangular matrix–vector multiplication, solution of a triangular system, or a rank-1 update of a rectangular matrix can be employed inside the outer loop. More generally, given a factorization $PA = LU$, two ways to evaluate A^{-1} are. as $A^{-1} = U^{-1} \times L^{-1} \times P$, and as the solution to $UA^{-1} = L^{-1} \times P$. These methods generally achieve different levels of efficiency on high-performance computers, and they propagate rounding errors in different ways. We concentrate in this chapter on the numerical stability, but comment briefly on performance issues.

The quality of an approximation $Y \approx A^{-1}$ can be assessed by looking at the right and left residuals, $AY - I$ and $YA - I$, and the forward error, $Y - A^{-1}$. Suppose we perturb $A \rightarrow A + \Delta A$ with $|\Delta A| \leq \epsilon |A|$; thus, we are making relative perturbations of size at most ϵ to the elements of A. If $Y = (A + \Delta A)^{-1}$ then $(A + \Delta A)Y = Y(A + \Delta A) = I$, so that

(13.1)

(13.2)

and, since $(A + \Delta A)^{-1} = A^{-1} - A^{-1}\Delta A A^{-1} + \mathcal{O}(\epsilon^2)$,

$$|A^{-1} - Y| \leq \epsilon |A^{-1}||A||A^{-1}| + O(\epsilon^2). \tag{13.3}$$

(Note that (13.3) can also be derived from (13.1) or (13.2).) The bounds (13.1)-(13.3) represent "ideal" bounds for a computed approximation Y to A^{-1}, if we regard ϵ as a small multiple of the unit roundoff u. We will show that, for triangular matrix inversion, appropriate methods do indeed achieve (13.1) or (13.2) (but not both) and (13.3).

It is important to note that neither (13.1), (13.2), nor (13.3) implies that $Y + \Delta Y = (A + \Delta A)^{-1}$ with $\|\Delta A\|_\infty \leq \epsilon \|A\|_\infty$ and $\|\Delta Y\|_\infty \leq \epsilon \|Y\|_\infty$, that is, Y need not be close to the inverse of a matrix near to A, even in the norm sense. Indeed, such a result would imply that both the left and right residuals are bounded in norm by $(2\epsilon + \epsilon^2)\|A\|_\infty \|Y\|_\infty$, and this is not the case for any of the methods we will consider.

To illustrate the latter point we give a numerical example. Define the matrix $A_n \in \mathbb{R}^{n \times n}$ as triu(qr(vand(n))), in MATLAB notation (vand is a routine from the Test Matrix Toolbox—see Appendix E); in other words, A_n is the upper triangular QR factor of the $n \times n$ Vandermonde matrix based on

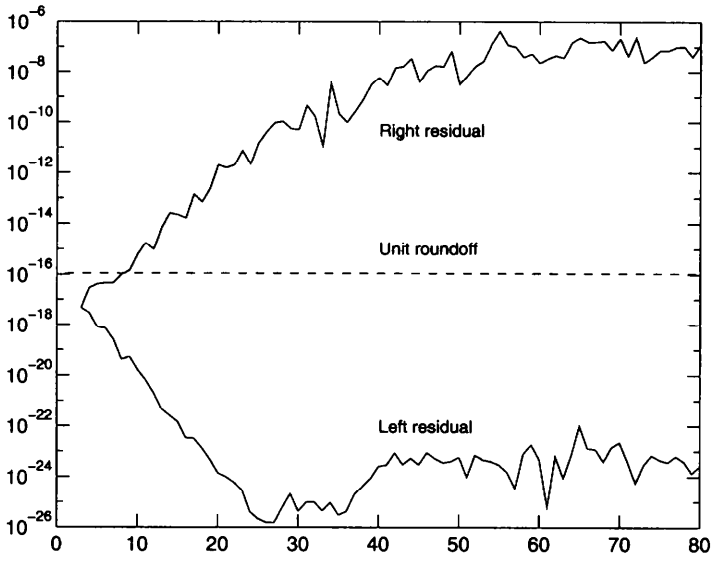

Figure 13.1. *Residuals for inverses computed by* MATLAB'S **INV** *function.*

equispaced points on [0, 1]. We inverted A_n, for $n = 1{:}80$, using MATLAB'S **INV** function, which uses GEPP. The left and right normwise relative residuals

$$\mathrm{res_L} = \frac{\|\widehat{X}A - I\|_\infty}{\|\widehat{X}\|_\infty \|A\|_\infty}, \qquad \mathrm{res_R} = \frac{\|A\widehat{X} - I\|_\infty}{\|A\|_\infty \|\widehat{X}\|_\infty},$$

are plotted in Figure 13.1. We see that while the left residual is always less than the unit roundoff, the right residual becomes large as n increases. These matrices are very ill conditioned (singular to working precision for $n \geq 20$), yet it is still reasonable to expect a small residual, and we will prove in §13.3.2 that the left residual must be small, independent of the condition number.

In most of this chapter we are not concerned with the precise values of constants (§13.4 is the exception); thus c_n denotes a constant of order n. To simplify the presentation we introduce a special notation. Let $A_i A_i \in \mathbb{R}^{m_i \times n_i}$, $i = 1{:}k$, be matrices such that the product $A_1 A_2 \ldots A_k$ is defined and let

$$p = \sum_{i=1}^{k-1} n_i.$$

Then $\Delta(A_1, A_2, \ldots, A_k) \in \mathbb{R}^{m_1 \times n_k}$ denotes a matrix bounded according to

$$|\Delta(A_1, A_2, \ldots, A_k)| \leq c_p u |A_1||A_2| \ldots |A_k|.$$

This notation is chosen so that if $\widehat{C} = fl(A_1 A_2 \ldots A_k)$, with the product evaluated in any order, then

$$\widehat{C} = A_1 A_2 \ldots A_k + \Delta(A_1, A_2, \ldots, A_k).$$

13.2. Inverting a Triangular Matrix

We consider the inversion of a lower triangular matrix $L \in \mathbb{R}^{n \times n}$, treating unblocked and blocked methods separately. We do not make a distinction between partitioned and block methods in this section. All the results in this and the next section are from Du Croz and Higham [322, 1992].

13.2.1. Unblocked Methods

We focus our attention on two "*j*" methods that compute L^{-1} a column at a time. Analogous "*i*" and "*k*" methods exist, which compute L^{-1} row-wise or use outer products, respectively, and we comment on them at the end of the section.

The first method computes each column of $X = L^{-1}$ independently, using forward substitution. We write it as follows, to facilitate comparison with the second method.

```
Method 1.
for j = 1:n
    x_jj = l_jj^{-1}
    X(j + 1:n,j) = -x_jj L(j + 1:n,j)
    Solve L(j + 1:n, j + 1:n)X(j + 1:n, j) = X(j + 1:n,j),
    by forward substitution.
end
```

In BLAS terminology, this method is dominated by n calls to the level-2 BLAS routine `xTRSV` (Triangular SolVe).

The second method computes the columns in the reverse order. On the *j*th step it multiplies by the previously computed inverse $L(j + 1{:}n, j + 1{:}n)^{-1}$ instead of solving a system with coefficient matrix $L(j + 1{:}n, j + 1{:}n)$.

```
Method 2.
for j = n:-1:1
    x_jj = l_jj^{-1}
    X(j + 1:n, j) = X(j + 1:n, j + 1:n)L(j + 1:n, j)
    X(j + 1:n, j) = -x_jj X(j + 1:n, j)
end
```

Method 2 uses n calls to the level-2 BLAS routine `xTRMV` (Triangular Matrix times Vector). On most high-performance machines `xTRMV` can be implemented to run faster than `xTRSV`, so Method 2 is generally preferable to Method 1 from the point of view of efficiency (see the performance figures at the end of §13.2.2). We now compare the stability of the two methods.

Theorem 8.5 shows that the jth column of the computed $\widehat{X}$ from Method 1 satisfies

$$(L + \Delta L_j)\widehat{x}_j = e_j, \quad |\Delta L_j| \le c_n u |L|.$$

It follows that we have the componentwise residual bound

$$|L\widehat{X} - I| \le c_n u |L||\widehat{X}| \tag{13.4}$$

and the componentwise forward error bound

$$|\widehat{X} - L^{-1}| \le c_n u |L^{-1}||L||\widehat{X}|. \tag{13.5}$$

Since $\widehat{X} = L^{-1} + O(u)$, (13.5) can be written as

$$|\widehat{X} - L^{-1}| \le c_n u |L^{-1}||L||L^{-1}| + O(u^2), \tag{13.6}$$

which is invariant under row and column scaling of L. If we take norms we obtain normwise relative error bounds that are either row or column scaling independent: from (13.6) we have

$$\frac{\|\widehat{X} - L^{-1}\|_\infty}{\|L^{-1}\|_\infty} \le c_n u \, \mathrm{cond}(L^{-1}) + O(u^2), \tag{13.7}$$

and the same bound holds with $\mathrm{cond}(L^{-1})$ replaced by $\mathrm{cond}(L)$.

Notice that (13.4) is a bound for the *right residual*, $LX - I$. This is because Method 1 is derived by solving $LX = I$. Conversely, Method 2 can be derived by solving $XL = I$, which suggests that we should look for a bound on the left residual for this method.

Lemma 13.1. *The computed inverse* $\widehat{X}$ *from Method 2 satisfies*

(13.8)

Proof. The proof is by induction on n, the case $n = 1$ being trivial. Assume the result is true for $n - 1$ and write

$$L = \begin{bmatrix} \alpha & 0 \\ y & M \end{bmatrix}, \qquad X = L^{-1} = \begin{bmatrix} \beta & 0 \\ z & N \end{bmatrix},$$

where $\alpha, \beta \in \mathbb{R}$, $y, z \in \mathbb{R}^{n-1}$ and $M, N \in \mathbb{R}^{(n-1)\times(n-1)}$. Method 2 computes the first column of X by solving $XL = I$ according to

$$\beta = a^{-1}, \qquad z = -\beta N y.$$

In floating point arithmetic we obtain

$$\widehat{\beta} = \alpha^{-1}(1+\delta), \qquad |\delta| \le u,$$
$$\widehat{z} = -\widehat{\beta}\widehat{N}y + \Delta(\widehat{\beta}, \widehat{N}, y).$$

Thus

$$\widehat{\beta}\alpha = 1 + \delta,$$
$$\widehat{z}\alpha + \widehat{N}y = -\delta\widehat{N}y + \alpha\Delta(\widehat{\beta}, \widehat{N}, y).$$

This may be written as

$$|\widehat{X}L - I|(1\colon n, 1) \le \begin{bmatrix} u \\ u|\widehat{N}||y| + c_n u(1+u)|\widehat{N}||y| \end{bmatrix}$$
$$\le c_n' u\big(|\widehat{X}||L|\big)(1\colon n, 1).$$

By assumption, the corresponding inequality holds for the $(2\colon n,\ 2\colon n)$ submatrices and so the result is proved. □

Lemma 13.1 shows that Method 2 has a left residual analogue of the right residual bound (13.4) for Method 1. Since there is, in general, no reason to choose between a small right residual and a small left residual, our conclusion is that Methods 1 and 2 have equally good numerical stability properties.

More generally, it can be shown that all three i, j, and k inversion variants that can be derived from the equations $LX = I$ produce identical rounding errors under suitable implementations, and all satisfy the same right residual bound; likewise, the three variants corresponding to the equation $XL = I$ all satisfy the same left residual bound. The LINPACK routine `xTRDI` uses a k variant derived from $XL = I$; the LINPACK routines `xGEDI` and `xPODI` contain analogous code for inverting an upper triangular matrix (but the LINPACK Users' Guide [307, 1979, Chaps. 1 and 3] describes a different variant from the one used in the code).

13.2.2. Block Methods

Let the lower triangular matrix $L \in \mathbb{R}^{n\times n}$ be partitioned in block form as

$$L = \begin{bmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ \vdots & & \ddots & \\ L_{N1} & \cdots & \cdots & L_{NN} \end{bmatrix}, \tag{13.9}$$

where we place no restrictions on the block sizes, other than to require the diagonal blocks to be square. The most natural block generalizations of Methods 1 and 2 are as follows. Here, we use the notation $L_{p:q,r:s}$ to denote the

submatrix comprising the intersection of block rows p to q and block columns r to s of L.

Method 1B.
for $j = 1{:}N$
  $X_{jj} = L_{jj}^{-1}$ (by Method 1)
  $X_{j+1:N,j} = -L_{j+1:N,j}X_{jj}$
  Solve $L_{j+1:N,j+1:N}X_{j+1:N,j} = X_{j+1:N,j}$,
  by forward substitution
end

Method 2B.
for $j = N{:}-1{:}1$
  $X_{jj} = L_{jj}^{-1}$ (by Method 2)
  $X_{j+1:N,j} = X_{j+1:N,j+1:N}L_{j+1:N,j}$
  $X_{j+1:N,j} = -X_{j+1:N,j}X_{jj}$
end

One can argue that Method 1B carries out the same arithmetic operations as Method 1, although possibly in a different order, and that it therefore satisfies the same error bound (13.4). For completeness, we give a direct proof.

Lemma 13.2. *The computed inverse* $\widehat{X}$ *from Method* 1B *satisfies*

$$|L\widehat{X} - I| \le c_n u|L||\widehat{X}|. \tag{13.10}$$

Proof. Equating block columns in (13.10), we obtain the N independent inequalities

$$|L\widehat{X}_{1:N,j} - I_{1:N,j}| \le c_n u|L||\widehat{X}_{1:N,j}|, \quad j = 1{:}N. \tag{13.11}$$

It suffices to verify the inequality with $j = 1$. Write

$$L = \begin{bmatrix} L_{11} & \\ L_{21} & L_{22} \end{bmatrix}, \qquad X = \begin{bmatrix} X_{11} & \\ X_{21} & X_{22} \end{bmatrix},$$

where $L_{11}, X_{11} \in \mathbb{R}^{r\times r}$ and L_{11} is the (1, 1) block in the partitioning of (13.9). X_{11} is computed by Method 1 and so, from (13.4),

$$|L_{11}\widehat{X}_{11} - I| \le c_r u|L_{11}||\widehat{X}_{11}| = c_r u\big(|L||\widehat{X}|\big)_{11}. \tag{13.12}$$

X_{21} is computed by forming $T = -L_{21}X_{11}$ and solving $L_{22}X_{21} = T$. The computed X_{21} satisfies

$$L_{22}\widehat{X}_{21} + \Delta(L_{22}, \widehat{X}_{21}) = -L_{21}\widehat{X}_{11} + \Delta(L_{21}, \widehat{X}_{11}).$$

Hence

$$|L_{21}\widehat{X}_{11} + L_{22}\widehat{X}_{21}| \le c_n u\big(|L_{21}||\widehat{X}_{11}| + |L_{22}||\widehat{X}_{21}|\big) = c_n u\big(|L||\widehat{X}|\big)_{21}. \tag{13.13}$$

Together, inequalities (13.12) and (13.13) are equivalent to (13.11) with $j = 1$, as required. □

We can attempt a similar analysis for Method 2B. With the same notation as above, X_{21} is computed as $X_{21} = -X_{2\,2}L_{2\,1}X_{11}$. Thus

$$\widehat{X}_{21} = -\widehat{X}_{22}L_{21}\widehat{X}_{11} + \Delta(\widehat{X}_{22}, L_{21}, \widehat{X}_{11}). \tag{13.14}$$

To bound the left residual we have to postmultiply by L_{11} and use the fact that X_{11} is computed by Method 2:

$$\widehat{X}_{21}L_{11} + \widehat{X}_{22}L_{21}(I + \Delta(\widehat{X}_{11}, L_{11})) = \Delta(\widehat{X}_{22}, L_{21}, \widehat{X}_{11})L_{11}.$$

This leads to a bound of the form

$$|\widehat{X}_{21}L_{11} + \widehat{X}_{22}L_{21}| \le c_n u|\widehat{X}_{22}||L_{21}||\widehat{X}_{11}||L_{11}|,$$

which would be of the desired form in (13.8) were it not for the factor $|\widehat{X}_{11}||L_{11}|$. This analysis suggests that the left residual is not guaranteed to be small. Numerical experiments confirm that the left and right residuals can be large simultaneously for Method 2B, although examples are quite hard to find [322, 1992]; therefore the method must be regarded as unstable when the block size exceeds 1.

The reason for the instability is that there are *two* plausible block generalizations of Method 2 and we have chosen an unstable one that does not carry out the same arithmetic operations as Method 2. If we perform a solve with L_{jj} instead of multiplying by X_{jj} we obtain the second variation, which is used by LAPACK's `xTRTRI`:

Method 2C.
for $j = N\text{:-}1\text{:}1$
  $X_{jj} = L_{jj}^{-1}$ (by Method 2)
  $X_{j+1:N,j} = X_{j+1:N,\,j+1:N}L_{j+1:N,\,j}$
  Solve $X_{j+1:N,j}L_{jj} = -X_{j+1:N,j}$ by back substitution.
end

For this method, the analogue of (13.14) is

$$\widehat{X}_{21}L_{11} + \Delta(\widehat{X}_{21}, L_{11}) = -\widehat{X}_{22}L_{21} + \Delta(\widehat{X}_{22}, L_{21})$$

which yields

$$|\widehat{X}_{21}L_{11} + \widehat{X}_{22}L_{21}| \le c_n u\big(|\widehat{X}_{21}||L_{11}| + |\widehat{X}_{22}||L_{21}|\big).$$

Hence Method 2C enjoys a very satisfactory residual bound.

Table 13.2. *Mflop rates for inverting a triangular matrix on a Cray 2.*

		$n = 128$	$n = 256$	$n = 512$	$n = 1024$
Unblocked:	Method 1	95	162	231	283
	Method 2	114	211	289	330
	k variant	114	157	178	191
Blocked:	Method 1B	125	246	348	405
(block size 64)	Method 2C	129	269	378	428
	k variant	148	263	344	383

Lemma 13.3. *The computed inverse* $\widehat{X}$ *from Method* 2C *satisfies*

$$|\widehat{X}L - I| \le c_n u |\widehat{X}||L|. \qquad \square$$

In summary, block versions of Methods 1 and 2 are available that have the same residual bounds as the point methods. However, in general, there is no guarantee that stability properties remain unchanged when we convert a point method to block form, as shown by Method 2B.

In Table 13.2 we present some performance figures for inversion of a lower triangular matrix on a Cray 2. These clearly illustrate the possible gains in efficiency from using block methods, and also the advantage of Method 2 over Method 1. For comparison, the performance of a k variant is also shown (both k variants run at the same rate). The performance characteristics of the i variants are similar to those of the j variants, except that since they are row oriented rather than column oriented, they are liable to be slowed down by memory-bank conflicts, page thrashing, or cache missing.

13.3. Inverting a Full Matrix by LU Factorization

Next, we consider four methods for inverting a full matrix $A \in \mathbb{R}^{n \times n}$, given an LU factorization computed by GEPP. We assume, without loss of generality, that there are no row interchanges. We write the *computed* LU factors as L and U. Recall that $A + \Delta A = LU$, with $|\Delta A| \le c_n u |L||U|$ (Theorem 9.3).

13.3.1. Method A

Perhaps the most frequently described method for computing $X = A^{-1}$ is the following one.

Method A.
for $j = 1{:}n$

Solve $Ax_J = e_j$
end

Compared with the methods to be described below, Method A has the disadvantages of requiring more temporary storage and of not having a convenient partitioned version. However, it is simple to analyse. From Theorem 9.4 we have

$$(A + \Delta A_j)\widehat{x}_j = e_j, \qquad |\Delta A_j| \le c_n' u |L||U|, \tag{13.15}$$

and so

$$|A\widehat{X} - I| \le c_n' u |L||U||\widehat{X}|. \tag{13.16}$$

This bound departs from the form (13. 1) only in that $|A|$ is replaced by its upper bound $|L||U| + O(u)$. The forward error bound corresponding to (13.16) is

$$|\widehat{X} - A^{-1}| \le c_n' u |A^{-1}||L||U||\widehat{X}|. \tag{13.17}$$

Note that (13.15) says that $\widehat{x}_j$ is the jth column of the inverse of a matrix close to A, but it is a different perturbation ΔA_j for each column. It is not true that $\widehat{X}$ itself is the inverse of a matrix close to A, unless A is well conditioned.

13.3.2. Method B

Next, we consider the method used by LINPACK'S `xGEDI`, LAPACK'S `xGETRI`, and MATLAB'S **INV** function.

Method B.
Compute U^{-1} and then solve for X the equation $XL = U^{-1}$.

To analyse this method we will assume that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices that obtains the columns of U^{-1} in the order 1 to n. Then the computed inverse $Xu \approx U^{-1}$ will satisfy the residual bound

$$|X_U U - I| \le c_n u |X_U||U|.$$

We also assume that the triangular solve from the right with L is done by back substitution. The computed X therefore satisfies $XL = Xu + \Delta(X, L)$ and so

$$\widehat{X}(A + \Delta A) = \widehat{X}LU = X_U U + \Delta(\widehat{X}, L)U.$$

This leads to the residual bound

$$\begin{aligned} |\widehat{X}A - I| &\le c_n u\big(|U^{-1}||U| + 2|\widehat{X}||L||U|\big) \\ &\le c_n' u |\widehat{X}||L||U|, \end{aligned} \tag{13.18}$$

which is the left residual analogue of (13.16). From (13.18) we obtain the forward error bound

$$|\widehat{X} - A^{-1}| \leq c_n' u |\widehat{X}||L||U||A^{-1}|.$$

Note that Methods A and B are equivalent, in the sense that Method A solves for X the equation $LUX = I$ while Method B solves $XLU = I$. Thus the two methods carry out analogous operations but in different orders. It follows that the methods must satisfy analogous residual bounds, and so (13.18) can be deduced from (13.16).

We mention in passing that the LINPACK manual states that for Method B a bound holds of the form $||AX - I|| \leq d_n u ||A||\ ||X||$ [307, 1979, p. 1.20]. This is incorrect, although counterexamples are rare; it is the left residual that is bounded this way, as follows from (13.18).

13.3.3. Method C

The next method that we consider is from Du Croz and Higham [322, 1992]. It solves the equation $UXL = I$, computing X a partial row and column at a time. To derive the method partition

$$X = \begin{bmatrix} x_{11} & x_{12}^T \\ x_{21} & X_{22} \end{bmatrix}, \qquad L = \begin{bmatrix} 1 & 0 \\ l_{21} & L_{22} \end{bmatrix}, \qquad U = \begin{bmatrix} u_{11} & u_{12}^T \\ 0 & U_{22} \end{bmatrix},$$

where the (1, 1) blocks are scalars, and assume that the trailing submatrix X_{22} is already known. Then the rest of X is computed according to

$$\begin{aligned} x_{21} &= -X_{22} l_{21}, \\ x_{12}^T &= -u_{12}^T X_{22}/u_{11}, \\ x_{11} &= 1/u_{11} - x_{12}^T l_{21}. \end{aligned}$$

The method can also be derived by forming the product $X = U^{-1} \times L^{-1}$ using the representation of L and U as a product of elementary matrices (and diagonal matrices in the case of U). In detail the method is as follows.

Method C.
for $k = n\text{:}-1\text{:}1$
$\quad X(k+1\text{:}n, k) = -X(k+1\text{:}n, k+1\text{:}n) L(k+1\text{:}n, k)$
$\quad X(k, k+1\text{:}n) = -U(k, k+1\text{:}n) X(k+1\text{:}n, k+1\text{:}n)/u_{kk}$
$\quad x_{kk} = 1/u_{kk} - X(k, k+1\text{:}n) L(k+1\text{:}n, k)$
end

The method can be implemented so that X overwrites L and U, with the aid of a work vector of length n (or a work array to hold a block row or column

in the partitioned case). Because most of the work is performed by matrix–vector (or matrix-matrix) multiplication, Method C is likely to be the fastest of those considered in this section on many machines. (Some performance figures are given at the end of the section.)

A straightforward error analysis of Method C shows that the computed $\widehat{X}$ satisfies

$$|U\widehat{X}L - I| \leq c_n u|U||\widehat{X}||L|. \tag{13.19}$$

We will refer to $U\widehat{X}L - I$ as a "mixed residual". From (13.19) we can obtain bounds on the left and right residual that are weaker than those in (13.18) and (13.16) by a factor $|U^{-1}||U|$ on the left or $|L||L^{-1}|$ on the right, respectively. We also obtain from (13.19) the forward error bound

$$|\widehat{X} - A^{-1}| \leq c_n u|U^{-1}||U||\widehat{X}||L||L^{-1}|,$$

which is (13.17) with $|A^{-1}|$ replaced by its upper bound $|U^{-1}||L^{-1}| + O(u)$ and the factors reordered.

The LINPACK routine `xSIDI` uses a special case of Method C in conjunction with the diagonal pivoting method to invert a symmetric indefinite matrix; see Du Croz and Higham [322, 1992] for details.

13.3.4. Method D

The next method is based on another natural way to form A^{-1} and is used by LAPACK'S `xPOTRI`, which inverts a symmetric positive definite matrix.

Method D.
Compute L^{-1} and U^{-1} and then form $A^{-1} = U^{-1} \times L^{-1}$.

The advantage of this method is that no extra workspace is needed; U^{-1} and L^{-1} can overwrite U and L, and can then be overwritten by their product. However, Method D is significantly slower on some machines than Methods B or C, because it uses a smaller average vector length for vector operations.

To analyse Method D we will assume initially that L^{-1} is computed by Method 2 (or Method 2C) and, as for Method B above, that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices. We have

$$\widehat{X} = X_U X_L + \Delta(X_U, X_L). \tag{13.20}$$

Since $A = LU - \Delta A$,

$$\widehat{X}A = X_U X_L LU - X_U X_L \Delta A + \Delta(X_U, X_L)A. \tag{13.21}$$

Rewriting the first term of the right-hand side using $X_L L = I + \Delta(X_L, L)$, and similarly for U, we obtain

$$\widehat{X}A - I = \Delta(X_U, U) + X_U\Delta(X_L, L)U - X_U X_L \Delta A + \Delta(X_U, X_L)A, \tag{13.22}$$

and so

$$\begin{aligned} |\widehat{X}A - I| &\le c_n' u\bigl(|U^{-1}||U| + 2|U^{-1}||L^{-1}||L||U| + |U^{-1}||L^{-1}||A|\bigr) \\ &\le c_n'' u|U^{-1}||L^{-1}||L||U|. \end{aligned} \tag{13.23}$$

This bound is weaker than (13.18), since $|\widehat{X}| \le |U^{-1}||L^{-1}| + O(u)$. Note, however, that the term $\Delta(X_U,X_L)A$ in the residual (13.22) is an unavoidable consequence of forming X_UX_L, and so the bound (13.23) is essentially the best possible.

The analysis above assumes that X_L and X_U both have small left residuals. If they both have small right residuals, as when they are computed using Method 1, then it is easy to see that a bound analogous to (13.23) holds for the right residual $A\widehat{X} - I$. If X_L has a small left residual and X_U has a small right residual (or vice versa) then it does not seem possible to derive a bound of the form (13.23). However, we have

$$|X_LL - I| = |L^{-1}(LX_L - I)L| \le |L^{-1}||LX_L - I||L|, \tag{13.24}$$

and since L is unit lower triangular with $|l_{ij}| \le 1$, we have $|(L^{-1})_{ij}| \le 2^{n-1}$, which places a bound on how much the left and right residuals of X_L can differ. Furthermore, since the matrices L from GEPP tend to be well conditioned $(\kappa_\infty(L) \ll n2^{n-1})$ and since our numerical experience is that large residuals tend to occur only for ill-conditioned matrices, we would expect the left and right residuals of XL almost always to be of similar size. We conclude that even in the "conflicting residuals" case, Method D will, in practice, usually satisfy (13.23) or its right residual counterpart, according to whether X_U has a small left or right residual respectively. Similar comments apply to Method B when U^{-1} is computed by a method yielding a small right residual.

These considerations are particularly pertinent when we consider Method D specialized to symmetric positive definite matrices and the Cholesky factorization $A = R^TR$. Now A^{-1} is obtained by computing $X_R = R^{-1}$ and then forming $A^{-1} = X_RX_R^T$; this is the method used in the LINPACK routine `xPODI` [307, 1979, Chap. 3]. If X_R has a small right residual then X_R^T has a small left residual, so in this application we naturally encounter conflicting residuals. Fortunately, the symmetry and definiteness of the problem help us to obtain a satisfactory residual bound. The analysis parallels the derivation of (13.23), so it suffices to show how to treat the term $X_RX_R^TR^TR$ (cf. (13.21)), where R now denotes the computed Cholesky factor. Assuming $RX_R = I + \Delta(R, X_R)$, and using (13.24) with L replaced by R, we have

$$\begin{aligned} X_RX_R^TR^TR &= X_R\bigl(I + \Delta(R,X_R)^T\bigr)R \\ &= I + F + X_R\Delta(R,X_R)^TR, \qquad |F| \le |R^{-1}||\Delta(R,X_R)||R|, \\ &= I + G, \end{aligned}$$

Table 13.3. *Mflop rates for inverting a full matrix on a Cray 2.*

		$n = 64$	$n = 128$	$n = 256$	$n = 512$
Unblocked:	Method B	118	229	310	347
	Method C	125	235	314	351
	Method D	70	166	267	329
Blocked:	Method B	142	259	353	406
(block size 64)	Method C	144	264	363	415
	Method D	70	178	306	390

and

$$|G| \le c_n u(|R^{-1}||R||R^{-1}||R| + |R^{-1}||R^{-T}||R^T||R|).$$

From the inequality $\| \, |B| \, \|_2 \le \sqrt{n}\|B\|_2$ for $B \in \mathbb{R}^{n\times n}$, together with $||A||_2 = \|R\|_2^2 + O(u)$, it follows that

$$\|G\|_2 \le 2n^2 c_n u \|A\|_2 \|A^{-1}\|_2,$$

and thus overall we have a bound of the form

$$\|\widehat{X}A - I\|_2 \le d_n u \|A\|_2 \|\widehat{X}\|_2.$$

Since $\widehat{X}$ and A are symmetric the same bound holds for the right residual.

13.3.5. Summary

In terms of the error bounds, there is little to choose between Methods A, B, C, and D. Numerical results reported in [322, 1992] show good agreement with the bounds. Therefore the choice of method can be based on other criteria, such as performance and the use of working storage. Table 13.3 gives some performance figures for a Cray 2, covering both blocked (partitioned) and unblocked forms of Methods B, C, and D.

On a historical note, Tables 13.4 and 13.5 give timings for matrix inversion on some early computing devices; times for two modern machines are given for comparison. The inversion methods used for the timings on the early computers in Table 13.4 are not known, but are probably methods from this section or the next.

13.4. Gauss–Jordan Elimination

Whereas Gaussian elimination (GE) reduces a matrix to triangular form by elementary operations, Gauss–Jordan elimination (GJE) reduces it all the way

Table 13.4. *Times (minutes and seconds) for inverting an n × n matrix. Source for DEUCE, Pegasus, and Mark* 1 *timings:* [181, 1981].

n	DEUCE (English Electric) 1955	Pegasus (Ferranti) 1956	Manchester Mark 1 1951	HP 48G Calculator 1993	Sun SPARC-station ELC 1991
8	20s	26s	–	4s	.004s
16	58s	2m 37s	–	18s	.01s
24	3m 36s	7m 57s	8m	48s	.02s
32	7m 44s	17m 52s	16m	–	.04s

Table 13.5. *Additional timings for inverting an n × n matrix.*

Machine	Year	n	Time	Reference
Aiken Relay Calculator	1948	38	59½ hours	[764, 1948]
IBM 602 Calculating Punch	1949	10	8 hours	[1053, 1949]
SEAC (National Bureau of Standards)	1954	49	3 hours	[1004, 1954]
Datatron	1957	80'	2½ hours	[753, 1957]
IBM 704	1957	115*	19m 30s	[320, 1957]

[a] Block tridiagonal matrix, using an inversion method designed for such matrices. asymmetric positive definite matrix.

to diagonal form. GJE is usually presented as a method for matrix inversion, but it can also be regarded as a method for solving linear equations. We will take the method in its latter form, since it simplifies the error analysis. Error bounds for matrix inversion are obtained by taking unit vectors for the right-hand sides.

At the kth stage of GJE, all off-diagonal elements in the kth column are eliminated, instead of just those below the diagonal, as in GE. Since the elements in the lower triangle (including the pivots on the diagonal) are identical to those that occur in GE, Theorem 9.1 tells us that GJE succeeds if all the leading principal submatrices of A are nonsingular. With no form of pivoting GJE is unstable in general, for the same reasons that GE is unstable. Partial and complete pivoting are easily incorporated.

Algorithm 13.4 (Gauss–Jordan elimination). This algorithm solves the linear system $Ax = b$, where $A \in \mathbb{R}^{n\times n}$ is nonsingular, by GJE with partial pivoting.

```
for k = 1:n
    Find r such that |a_rk| = max_{i>k} |a_ik|.
    A(k, k:n) ↔ A(r, k:n), b(k) ↔ b(r) % Swap rows k and r.
    row_ind = [1:k − 1, k + 1:n] % Row indices of elements to eliminate.
    m = A(row_ind,k)/A(k,k) % Multipliers.
    A(row_ind,k:n) = A(row_ind,k:n) − m*A(k,k:n)
    b(row_ind) = b(row_ind) − m*b(k)
end
x_i = b_i/a_ii, i = 1:n
```

Cost: n^3 flops.

The numerical stability properties of GJE are rather subtle and error analysis is trickier than for GE. An observation that simplifies the analysis is that we can consider the algorithm as comprising two stages. The first stage is identical to GE and forms $M_{n-1}M_{n-2} \ldots M_1 A = U$, where U is upper triangular. The second stage reduces U to diagonal form by elementary operations:

$$N_n N_{n-1} \ldots N_2 U = D, \qquad N_k = I - n_k e_k^T, \quad e_i^T n_k = 0,\ i \geq k.$$

The solution x is formed as $x = D_{-1} N_n \ldots N_2 y$, where $y = M_{n-1} \ldots M_2 b$. The rounding errors in the first stage are precisely the same as those in GE, so it suffices to consider the second stage. We will assume, for simplicity, that there are no row interchanges. As with GE, this is equivalent to assuming that all row interchanges are done at the start of the algorithm.

Define $U_{k+1} = N_k \ldots N_2 U$ (so $U_2 = U$) and note that N_k and U_k have the forms

$$U_k = \begin{bmatrix} D_{k-1} & [v_k \ \ W_k] \\ 0 & Z_k \end{bmatrix}, \quad D_{k-1} \in \mathbb{R}^{(k-1)\times(k-1)}, \text{ diagonal},$$
$$N_k = \begin{bmatrix} I_{k-1} & [n_k \ \ 0] \\ 0 & I_{n-k+1} \end{bmatrix},$$

where $n_k = [-u_{1k}/u_{kk}, \ldots, -u_{k-1,k}/u_{kk}]^T$. The computed matrices obviously satisfy

$$\widehat{U}_{k+1} = \widehat{N}_k \widehat{U}_k + \Delta_k, \tag{13.25a}$$
$$\Delta_k = \begin{bmatrix} 0_{k-1} & \Delta^{(k)} \\ 0 & 0_{n-k+1} \end{bmatrix}, \quad |\Delta_k| \leq \gamma_3 |\widehat{N}_k||\widehat{U}_k|, \tag{13.25b}$$

(to be precise, $\widehat{N}_k$ is defined as N_k but with $(\widehat{N}_k)_{ik} = -\widehat{u}_{1k}/\widehat{u}_{kk}$). Similarly, with $x_{k+1} = N_k \ldots N_2 y$, we have

$$\widehat{x}_{k+1} = \widehat{N}_k \widehat{x} + f_k, \quad e_i^T f_k = 0,\ i \geq k, \quad |f_k| \leq \gamma_3 |\widehat{N}_k||\widehat{x}_k|. \tag{13.26}$$

Because of the structure of $\widehat{N}_k$, Δ_k, and f_k, we have the useful property that

$$\widehat{N}_i \Delta_j = \Delta_j, \quad i \geq j, \qquad \widehat{N}_i f_j = f_j, \quad i \geq j.$$

Without loss of generality, we now assume that the final diagonal matrix D is the identity (i.e., the pivots are all 1); this simplifies the analysis a little and has a negligible effect on the final bounds. Thus (13.25) and (13.26) yield

$$I = \widehat{U}_{n+1} = \widehat{N}_n \dots \widehat{N}_2 U + \sum_{k=2}^{n} \Delta_k, \tag{13.27}$$

$$\widehat{x} = \widehat{x}_{n+1} = \widehat{N}_n \dots \widehat{N}_2 y + \sum_{k=2}^{n} f_k. \tag{13.28}$$

Now

$$\begin{aligned} |\Delta_k| &\leq \gamma_3 |\widehat{N}_k| |\widehat{U}_k| \leq \gamma_3 |\widehat{N}_k| |\widehat{N}_{k-1}\widehat{U}_{k-1} + \Delta_{k-1}| \\ &\leq \gamma_3(1+\gamma_3) |\widehat{N}_k| |\widehat{N}_{k-1}| |\widehat{U}_{k-1}| \\ &\leq \cdots \leq \gamma_3(1+\gamma_3)^{k-2} |\widehat{N}_k| \dots |\widehat{N}_2| |U|, \end{aligned}$$

and, similarly,

$$|f_k| \leq \gamma_3(1+\gamma_3)^{k-2} |\widehat{N}_k| \dots |\widehat{N}_2| |y|.$$

But defining $\tilde{n}_k = [n_k^T \ 0]^T \in \mathbb{R}^n$, we have

$$\begin{aligned} |\widehat{N}_k| \dots |\widehat{N}_2| &= I + |\tilde{n}_k| e_k^T + \cdots + |\tilde{n}_2| e_2^T \\ &\leq |\widehat{N}_n| \dots |\widehat{N}_2| = |\widehat{N}_n \dots \widehat{N}_2| =: |X|, \end{aligned}$$

where $X = U^{-1} + O(u)$ (by (13.27)). Hence

$$\begin{aligned} \sum_{k=2}^{n} |\Delta_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2} |X| |U|, \\ \sum_{k=2}^{n} |f_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2} |X| |y|. \end{aligned}$$

Combining (13.27) and (13.28) we have, for the solution of $Ux = y$,

$$\widehat{x} = \left(I - \sum_{k=2}^{n} \Delta_k\right) U^{-1} y + \sum_{k=2}^{n} f_k = \left(I - \sum_{k=2}^{n} \Delta_k\right) x + \sum_{k=2}^{n} f_k,$$

which gives the componentwise forward error bound

$$|x - \widehat{x}| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2} |X| (|U||x| + |y|). \tag{13.29}$$

Table 13.6. *Gauss-Jordan elimination for $U_x = b$.*

n	$\eta_{U,b}(\widehat{x})$	$\kappa_\infty(U)u$
16	2.0e-14	5.8e-11
32	6.4e-10	7.6e-6
64	1.7e-2	6.6e4

This is an excellent forward error bound: it says that the error in $\widehat{x}$ is no larger than the error we would expect for an approximate solution with a tiny componentwise relative backward error. In other words, the forward error is bounded in the sane way as for substitution. However, we have not, and indeed cannot, show that the method is backward stable. The best we can do is to derive from (13.27) and (13.28) the result

$$(U + \Delta U)\widehat{x} = y + \Delta y, \tag{13.30a}$$

$$|\Delta U| \le (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||U|, \tag{13.30b}$$

$$|\Delta y| \le (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||y|, \tag{13.30C}$$

using $\widehat{N}_2^{-1}\ldots\widehat{N}_n^{-1} = U + O(u)$. These bounds show that $\widehat{x}$ has a normwise backward error bounded approximately by $\kappa_\infty(U)(n-1)\gamma_3$. Hence the backward error can be guaranteed to be small only if U is well conditioned. This agrees with the comments of Peters and Wilkinson [828, 1975] that "the residuals corresponding to the Gauss-Jordan solution are often larger than those corresponding to back-substitution by a factor of order κ."

A numerical example helps to illustrate the results. We take U to be the upper triangular matrix with 1s on the diagonal and – 1s everywhere above the diagonal (U = $U(1)$ from (8.2)). This matrix has condition number $\kappa_\infty(U) = n2^{n-1}$. Table 13.6 reports the normwise relative backward error $\eta_{U,b}(\widehat{x}) = \|U\widehat{x} - b\|_\infty/(\|U\|_\infty\|\widehat{x}\|_\infty + \|b\|_\infty)$ (see (7.2)) for $b = U_x$, where $x = e/3$. Clearly, GJE is backward unstable for these matrices—the backward errors show a dependence on $\kappa_\infty(U)$. However, the relative distance between $\widehat{x}$ and the computed solution from substitution (not shown in the table) is less than $\kappa_\infty(U)u$, which shows that the forward error is bounded by $\kappa_\infty(U)u$, confirming (13.29).

By bringing in the error analysis for the reduction to upper triangular form, we obtain an overall result for GJE.

Theorem 13.5. *Suppose GJE successfully computes an approximate solution $\widehat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n\times n}$ is nonsingular. Then*

$$|b - A\widehat{x}| \le 8nu|\widehat{L}||\widehat{U}||\widehat{U}^{-1}||\widehat{U}||\widehat{x}| + O(u^2), \tag{13.31}$$

$$|x - \widehat{x}| \le \left(2n|A^{-1}||\widehat{L}||\widehat{U}| + 6n(|\widehat{U}^{-1}||\widehat{U}|)^2\right)|\widehat{x}| + O(u^2), \tag{13.32}$$

where $A \approx \widehat{L}\widehat{U}$ *is the factorization computed by GE.*

Proof. For the first stage (which is just GE), we have $A + \Delta A_1 = \widehat{L}\widehat{U}$, $|\Delta A_1| \le \gamma_n|\widehat{L}||\widehat{U}|$, and $(\widehat{L} + \Delta L)\widehat{y} = b$, $|\Delta L| \le \gamma_n|L|$, by Theorems 9.3 and 8.5.

Using (13.30), we obtain

$$(\widehat{L} + \Delta L)(\widehat{U} + \Delta U)\widehat{x} = (\widehat{L} + \Delta L)(\widehat{y} + \Delta y) = b + (\widehat{L} + \Delta L)\Delta y,$$

or $A\widehat{x} = b - r$, where

$$r = (\Delta A_1 + \widehat{L}\Delta U + \Delta L\widehat{U} + \Delta L\Delta U)\widehat{x} - (\widehat{L} + \Delta L)\Delta y. \tag{13.33}$$

The bounds (13.31) and (13.32) follow easily on using (13.30). □

Theorem 13.5 shows that the stability of GJE depends not only on the size of $|L||U|$ (as in GE), but also on the condition of U. The term $|\widehat{U}^{-1}||\widehat{U}||\widehat{x}|$ is an upper bound for $|\widehat{x}|$, and if this bound is sharp then the residual bound is very similar to that for LU factorization. Note that for partial pivoting we have $\kappa_\infty(\widehat{U}) \le n2^{n-1}\kappa_\infty(A) + O(u)$.

The bounds in Theorem 13.5 have the pleasing property that they are invariant under row or column scaling of A, though of course if we are using partial pivoting then row scaling can change the pivots and alter the bound.

As mentioned earlier, to obtain bounds for matrix inversion we simply take b to be each of the unit vectors in turn. For example, the residual bound becomes

$$|A\widehat{X} - I| \le 8nu|\widehat{L}||\widehat{U}||\widehat{U}^{-1}||\widehat{U}||\widehat{X}| + O(u^2).$$

For the special case of symmetric positive definite matrices, an informative normwise result follows from Theorem 13.5. We make the natural assumption that symmetry is exploited in the elimination.

Corollary 13.6. *Suppose GJE successfully computes an approximate solution* $\widehat{x}$ *to* $Ax = b$, *where* $A \in \mathbb{R}^{n\times n}$ *is symmetric positive definite. Then*

$$\|b - A\widehat{x}\|_2 \le 8n^3u\kappa_2(A)^{1/2}\|A\|_2\|x\|_2 + O(u^2),$$
$$\frac{\|x - \widehat{x}\|_2}{\|x\|_2} \le 8n^3u\kappa_2(A) + O(u^2),$$

where $A \approx \widehat{L}\widehat{U}$ *is the factorization computed by symmetric GE.*

Proof. By Theorem 9.3 we gave $A + \Delta A = \widehat{L}\widehat{U}$, where ΔA is symmetric and satisfies $|\Delta A| \le \gamma_n|\widehat{L}||\widehat{U}|$. Defining $D = \mathrm{diag}(\widehat{U})^{1/2}$ we have, by

symmetry, $A + \Delta A = \widehat{L}D \cdot D^{-1}\widehat{U} \equiv R^T R$. Hence

$$\begin{aligned} \| \, |\widehat{U}^{-1}||\widehat{U}| \, \|_2 &= \| \, |R^{-1}D^{-1}||DR| \, \|_2 \\ &= \| \, |R^{-1}||R| \, \|_2 \\ &\le n\kappa_2(R) = n\kappa_2(A + \Delta A)^{1/2} \\ &= n\kappa_2(A)^{1/2} + O(u). \end{aligned}$$

Furthermore, it is staightforward to show that $\| \, |\widehat{L}||\widehat{U}| \, \|_2 = \| \, |R^T||R| \, \|_2 \le n(1-\gamma_n)^{-1}\|A\|_2$. The required bounds follow.

Corollary 13.6 shows that GJE is forward stable for symmetric positive definite matrices, but it bounds the backward error only by a multiple of $\kappa_2(A)^{1/2}$. Numerical experiments show that the backward error is usually much less than $\kappa_2(A)^{1/2}u$, but (very ill-conditioned) matrices can certainly be found for which the backward error is many order of magnitude larger than u. Hence GJE is not backward stable even for symmetric positive definite matrices.

13.5. The Determinant

It may be too optimistic to hope that determinants will fade out of the mathematical picture in a generation; their notation alone is a thing of beauty to those who can appreciate that sort of beauty.

— E. T. BELL, *Review of "Contributions to the History of Determinants, 1900–1920", by Sir Thomas Muir (1931)*

Like the matrix inverse, the determinant is a quantity that rarely needs to be computed. The common fallacy that the determinant is a measure of ill conditioning is displayed by the observation that if $Q \in \mathbb{R}^{n\times n}$ is orthogonal then $\det(aQ) = a^n\det(Q) = \pm a^n$, which can be made arbitrarily small or large despite the fact that aQ is perfectly conditioned. Of course, we could normalize the matrix before taking its determinant and define, for example,

$$\psi(A) := \frac{1}{\det(D^{-1}A)} = \frac{\det(D)}{\det(A)}, \qquad D = \operatorname{diag}(\|A(i,:)\|_2),$$

where $D^{-1}A$ has rows of unit 2-norm. This function ψ is called the Hadamard condition number by Birkhoff [99, 1975], because Hadamard's determinantal inequality (see Problem 13.11) implies that $\psi(A) \ge 1$, with equality if and only if A has mutually orthogonal rows. Unless A is already row equilibrated (see Problem 13.13), $\psi(A)$ does not relate to the conditioning of linear systems in any straightforward way.

As good a way as any to compute the determinant of a general matrix $A \in \mathbb{R}^{n\times n}$ is via GEPP. If $PA = LU$ then

$$\det(A) = \det(P)^{-1} \det(U) = (-1)^r u_{11} \ldots u_{nn}, \tag{13.34}$$

where r is the number of row interchanges during the elimination. If we use (13.34) then, barring underflow and overflow, the computed determinant $\widehat{d} = fl(\det(A))$ satisfies

$$\widehat{d} = \det(\widehat{U})(1+\theta_n),$$

where $|\theta_n| \le \gamma_n$, so we have a tiny relative perturbation of the exact determinant of a perturbed matrix $A + \Delta A$, where ΔA is bounded in Theorem 9.3. In other words, we have almost the right determinant for a slightly perturbed matrix (assuming the growth factor is small).

However, underflow and overflow in computing the determinant are quite likely, and the determinant itself may not be a representable number. One possibility is to compute $\log|\det(A)| = \sum_{i=1}^{n} \log|u_{ii}|$; as pointed out by Forsythe and Moler [396, 1967, p. 55], however, the computed sum may be inaccurate due to cancellation. Another approach, used in LINPACK, is to compute and represent $\det(A)$ in the form $y \times 10^e$, where $1 \le |y| < 10$ and e is an integer exponent.

13.5.1. Hyman's Method

In §1.16 we analysed the evaluation of the determinant of a Hessenberg matrix H by GE. Another method for computing $\det(H)$ is Hyman's method [594, 1957], which is superficially similar to GE but algorithmically different. Hyman's method has an easy derivation in terms of LU factorization that leads to a very short error analysis. Let $H \in \mathbb{R}^{n\times n}$ bean unreduced upper Hessenberg matrix ($h_{i+1,i} \neq 0$ for all i) and write

$$H = \begin{bmatrix} h^T & \eta \\ T & y \end{bmatrix}, \qquad H_1 = \begin{bmatrix} T & y \\ h^T & \eta \end{bmatrix}, \qquad h, y \in \mathbb{R}^{n-1},\ \eta \in \mathbb{R}.$$

The matrix H_1 is H with the first row cyclically permuted to the bottom, so $\det(H_1) = (-1)^{n-1} \det(H)$. Since T is a nonsingular upper triangular matrix, we have the LU factorization

$$H_1 = \begin{bmatrix} I & 0 \\ h^T T^{-1} & 1 \end{bmatrix} \begin{bmatrix} T & y \\ 0 & \eta - h^T T^{-1} y \end{bmatrix} \equiv LU, \tag{13.35}$$

from which we obtain $\det(H_1) = \det(T)(\eta - h^T T^{-1} y)$. Therefore

$$\det(H) = (-1)^{n-1} \det(T)(\eta - h^T T^{-1} y). \tag{13.36}$$

Hyman's method consists of evaluating (13.36) in the natural way: by solving the triangular system $Tx = y$ by back substitution, then forming $\eta - h^T x$ and its product with $\det(T) = h_{21}h_{32} \ldots h_{n,n-1}$.

For the error analysis we assume that no underflows or overflows occur. From the backward error analysis for substitution (Theorem 8.5) and for an inner product ((3.4)) we have immediately, on defining $\mu := \eta - h^T T^{-1} y$,

$$\widehat{\mu} = \big(\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y\big)(1 + \delta),$$

where

$$|\Delta h| \le \gamma_{n-1}|h|, \quad |\Delta T| \le \gamma_{n-1}|T|, \quad |\delta| \le u.$$

Since $fl(\det(T)) = \det(T)(1 + \delta_1) \ldots (1 + \delta_{n-1})$, $|\delta_i| \le u$, the computed determinant satisfies

$$\widehat{d} := fl(\det(H)) = (1 + \theta_n)\det(T)\big(\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y\big),$$

where $|\theta_n| \le \gamma_n$. This is not a backward error result, because only one of the two Ts in this expression is perturbed. However, we can write $\det(T) = \det(T + \Delta T)(1 + \theta_{(n-1)2})$, so that

$$\widehat{d} = \det(T + \Delta T)\big(\eta(1 + \theta_{n^2-n+1}) - (h + \Delta h)^T (T + \Delta T)^{-1} y(1 + \theta_{n^2-n+1})\big).$$

We conclude that the computed determinant is the exact determinant of a perturbed Hessenberg matrix in which each element has undergone a relative perturbation not exceeding $\gamma_{n^2-n+1} \approx n^2 u$, which is a very satisfactory result. In fact, the constant γ_{n^2-n+1} may be reduced to γ_{2n-1} by a slightly modified analysis; see Problem 13.14.

13.6. Notes and References

Classic references on matrix inversion are Wilkinson's paper *Error Analysis of Direct Methods of Matrix Inversion* [1085, 1961] and his book *Rounding Errors in Algebraic Processes* [1088, 1963]. In discussing Method 1 of §13.2.1, Wilkinson says *"The residual of X as a left-hand inverse may be larger than the residual as a right-hand inverse by a factor as great as* $||L||\,||L^{-1}||$ *. . .* We are asserting that the computed X is *almost invariably* of such a nature that $XL - I$ is equally small" [1088, 1963, p. 107]. Our experience concurs with the latter statement. Triangular matrix inversion provides a good example of the value of rounding error analysis: it helps us identify potential instabilities, even if they are rarely manifested in practice, and it shows us what kind of stability is guaranteed to hold.

In [1085, 1961] Wilkinson identifies various circumstances in which triangular matrix inverses are obtained to much higher accuracy than the bounds

of this chapter suggest. The results of §8.2 provide some insight. For example, if T is a triangular M-matrix then, as noted after Corollary 8.10, its inverse is computed to high relative accuracy, no matter how ill conditioned L may be.

Sections 13.2 and 13.3 are based closely on Du Croz and Higham [322, 1992].

Method D in §13.3.4 is used by the Hewlett-Packard HP-15C calculator, for which the method's lack of need for extra storage is an important property [523, 1982].

Higham [560, 1995] gives error analysis of a divide-and-conquer method for inverting a triangular matrix that has some attractions for parallel computation.

GJE is an old method. It was discovered independently by the geodesist Wilhelm Jordan (1842-1899) (not the mathematician Camille Jordan (1838-1922)) and B.-I. Clasen [12, 1987].

An Algol routine for inverting positive definite matrices by GJE was published in the *Handbook for Automatic Computation* by Bauer and Reinsch [83, 1971]. As a means of solving a single linear system, GJE is 50% more expensive than GE when cost is measured in flops; the reason is that GJE takes $O(n^3)$ flops to solve the upper triangular system that GE solves in n^2 flops. However, GJE has attracted interest for vector computing because it maintains full vector lengths throughout the algorithm. Hoffmann [577, 1987] found that it was faster than GE on the CDC Cyber 205, a now-defunct machine with a relatively large vector startup overhead.

Turing [1027, 1948] gives a simplified analysis of the propagation of errors in GJE, obtaining a forward error bound proportional to $\kappa(A)$. Bauer [80, 1966] does a componentwise forward error analysis of GJE for matrix inversion and obtains a relative error bound proportional to $\kappa_\infty(A)$ for symmetric positive definite A. Bauer's paper is in German and his analysis is not easy to follow. A summary of Bauer's analysis (in English) is given by Meinguet [746, 1969].

The first thorough analysis of the stability of GJE was by Peters and Wilkinson [828, 1975]. Their paper is a paragon of rounding error analysis. Peters and Wilkinson observe the connection with GE, then devote their attention to the "second stage" of GJE, in which $Ux = y$ is solved. They show that each component of x is obtained by solving a lower triangular system, and they deduce that, for each i, $(U + \Delta U_i)x^{(i)} = y + \Delta y_i$, where $|\Delta U_i| \le \gamma_n|U|$ and $|\Delta y_i| \le \gamma_n|y|$, and where the ith component of $x^{(i)}$ is the ith component of $\widehat{x}$. They then show that $\widehat{x}$ has relative error bounded by $2n\kappa_\infty(U)u + O(u^2)$, but that $\widehat{x}$ does not necessarily have a small backward error. The more direct approach used in our analysis is similar to that of Dekker and Hoffmann [276, 1989], who give a normwise analysis of a variant of GJE that uses row pivoting (elimination across rows) and column interchanges. Our componentwise

bounds (13.29)–(13.32) are new.

Goodnight [473, 1979] describes the use of GJE in statistics for solving least squares problems.

Error analysis of Hyman's method is given by Wilkinson [1084, 1960], [1088, 1963, pp. 147-154], [1089, 1965, pp. 426-431]. Although it dates from the 1950s, Hyman's method is not obsolete: it has found use in methods designed for high-performance computers; see Ward [1065, 1976], Li and Zeng [701, 1992], and Dubrulle and Golub [324, 1994].

13.6.1. LAPACK

Routine `xGETRI` computes the inverse of a general square matrix by Method B using an LU factorization computed by `xGETRF`. The corresponding routine for a symmetric positive definite matrix is `xPOTRI`, which uses Method D, with a Cholesky factorization computed by `xPOTRF`. Inversion of a symmetric indefinite matrix is done by `xSYTRI`. Triangular matrix inversion is done by `xTRTRI`, which uses Method 2C. None of the LAPACK routines compute the determinant, but it is easy for the user to evaluate it after computing an LU factorization.

Problems

13.1. Reflect on this cautionary tale told by Acton [4, 1970, p. 246].

"It was 1949 in Southern California. Our computer was a very new CPC (model 1, number 1) —a 1-second-per-arithmetic-operation clunker that was holding the computational fort while an early electronic monster was being coaxed to life in an adjacent room, From a nearby aircraft company there arrived one day a 16×16 matrix of 10-digit numbers whose inverse was desired ... We labored for two days and, after the usual number of glitches that accompany any strange procedure involving repeated handling of intermediate decks of data cards, we were possessed of an inverse matrix. During the checking operations . . . it was noted that, to eight significant figures, the inverse was the transpose of the original matrix! A hurried visit to the aircraft company to explore the source of the matrix revealed that each element had been laboriously hand computed from some rather simple combinations of sines and cosines of a common angle. It took about 10 minutes to prove that the matrix was, indeed, orthogonal!"

13.2. Rework the analysis of the methods of §13.2.2 using the assumptions (12.3) and (12.4), thus catering for possible use of fast matrix multiplication techniques.

13.3. Show that for any nonsingular matrix A,

$$\kappa(A) \geq \max\left\{ \frac{\|AX - I\|}{\|XA - I\|}, \frac{\|XA - I\|}{\|AX - I\|} \right\}.$$

This inequality shows that the left and right residuals of X as an approximation to A^{-1} can differ greatly only if A is ill conditioned.

13.4. (Mendelssohn [748, 1956]) Find parametrized 2×2 matrices A and X such that the ratio $||AX - I||/||XA - I||$ can be made arbitrarily large.

13.5. Let X and Y be approximate inverses of $A \in \mathbb{R}^{n\times n}$ that satisfy

$$|A\widehat{X} - I| \leq u|A||\widehat{X}| \quad \text{and} \quad |\widehat{Y}A - I| \leq u|\widehat{Y}||A|,$$

and let $\widehat{x} = fl(\widehat{X}b)$ and $\widehat{y} = fl(\widehat{Y}b)$. Show that

$$|A\widehat{x} - b| \leq \gamma_{n+1}|A||\widehat{X}||b| \quad \text{and} \quad |A\widehat{y} - b| \leq \gamma_{n+1}|A||\widehat{Y}||A||x|.$$

Derive forward error bounds for $\widehat{x}$ and $\widehat{y}$. Interpret all these bounds.

13.6. What is the relation between the matrix on the front cover of the *LAPACK Users' Guide* [17, 1995] and that on the back cover? Answer the same question for the *LINPACK Users' Guide* [307, 1979].

13.7. Show that for any matrix having a row or column of 1s, the elements of the inverse sum to 1.

13.8. Let $X = A + iB \in \mathbb{C}^{n\times n}$ be nonsingular. Show that X^{-1} can be expressed in terms of the inverse of the real matrix of order $2n$,

$$Y = \begin{bmatrix} A & -B \\ B & A \end{bmatrix}.$$

Show that if X is Hermitian positive definite then Y is symmetric positive definite. Compare the economics of real versus complex inversion.

13.9. For a given nonsingular $A \in \mathbb{R}^{n\times n}$ and $X \approx A^{-1}$, it is interesting to ask what is the smallest ε such that $X + \Delta X = (A + \Delta A)^{-1}$ with $||\Delta X|| \leq \epsilon\|X\|$ and $\|\Delta A\| \leq \epsilon\|A\|$. We require $(A + \Delta A)(X + \Delta X) = I$, or

$$A\Delta X + \Delta A X + \Delta A \Delta X = I - AX.$$

It is reasonable to drop the second-order term, leaving a generalized Sylvester equation that can be solved using singular value decompositions of A and X (cf. §15.2). Investigate this approach computationally for a variety of A and methods of inversion.

13.10. For a nonsingular $A \in \mathbb{R}^{n\times n}$ and given integers i and j, under what conditions is $\det(A)$ independent of a_{ij}? What does this imply about the suitability of $\det(A)$ for measuring conditioning?

13.11. Prove Hadamard's inequality for $A \in \mathbb{C}^{n\times n}$:

$$|\det(A)| \le \prod_{k=1}^{n} \|a_k\|_2,$$

where $a_k = A(:, k)$. When is there equality? (Hint: use the QR factorization.)

13.12. (a) Show that if $A^T = QR$ is a QR factorization then the Hadamard condition number $\psi(A) = \prod_{i=1}^{n} \rho_i/|r_{ii}|$, where $\rho_i = \|R(:, i)\|_2$. (b) Evaluate $\psi(A)$ for $A = U(1)$ (see (8.2)) and for the Pei matrix $A = (a - 1)I + ee^T$.

13.13. (Guggenheimer, Edelman, and Johnson [486, 1995]) (a) Prove that for a nonsingular $A \in \mathbb{R}^{n\times n}$,

$$\kappa_2(A) < \frac{2}{|\det(A)|}\left(\frac{\|A\|_F}{n^{1/2}}\right)^n.$$

(Hint: apply the arithmetic-geometric mean inequality to the n numbers $\sigma_1^2/2, \sigma_1^2/2, \sigma_2^2, \ldots, \sigma_{n-1}^2$, where the σ_i are the singular values of A.) (b) Deduce that if A has rows of unit 2-norm then

$$\kappa_2(A) < \frac{2}{|\det(A)|} = 2\psi(A),$$

where ψ is the Hadamard condition number.

13.14. Show that Hyman's method for computing $\det(H)$, where $H \in \mathbb{R}^{n\times n}$ is an unreduced upper Hessenberg matrix, computes the exact determinant of $H + \Delta H$ where $|\Delta H| \le \gamma_{2n-1}|H|$, barring underflow and overflow. What is the effect on the error bound of a diagonal similarity transformation $H' \rightarrow D^{-1}HD$, where $D = \mathrm{diag}(d_i)$, $d_i \neq 0$?

13.15. What is the condition number of the determinant?

13.16. (RESEARCH PROBLEM) Obtain backward and forward error bounds for GJE applied to a diagonally dominant matrix. Peters and Wilkinson [828, 1975] state that "it is well known that Gauss–Jordan is stable" for a diagonally dominant matrix, but a proof does not seem to have been published.

Chapter 14
Condition Number Estimation

Most of LAPACK'S condition numbers and error bounds are based on estimated condition numbers . . .
The price one pays for using an estimated rather than an exact condition number is occasional (but very rare) underestimates of the true error; years of experience attest to the reliability of our estimators, although examples where they badly underestimate the error can be constructed.

— E. ANDERSON et al,, *LAPACK Users' Guide, Release 2.0 (1995)*

The importance of the counter-examples is that they make clear that any effort toward proving that the algorithms always produce useful estimations is fruitless. It may be possible to prove that the algorithms produce useful estimations in certain situations, however, and this should be pursued. An effort simply to construct more complex algorithms is dangerous.

— A. K. CLINE and R. K. REW, *A Set of Counter-Examples to Three Condition Number Estimators (1983)*

Singularity is almost invariably a clue.

— SIR ARTHUR CONAN DOYLE, *The Boscombe Valley Mystery "(1892)*

14.1. How to Estimate Componentwise Condition Numbers

When bounding the forward error of a computed solution to a linear system we would like to obtain the bound with an order of magnitude less work than is required to compute the solution. For a dense $n \times n$ system, where the solution process usually requires $O(n^3)$ operations, we need to compute the bound in $O(n^2)$ operations. An estimate of the bound that is correct to within a factor 10 is usually acceptable, because it is the magnitude of the error that is of interest, not its precise value.

In the perturbation theory of Chapter 7 for linear equations we obtained perturbation bounds that involve the condition numbers

$$\kappa_{E,f}(A,x) = \frac{\|A^{-1}\|\,\|f\|}{\|x\|} + \|A^{-1}\|\,\|E\|,$$

$$\operatorname{cond}_{E,f}(A,x) = \frac{\| \,|A^{-1}|f + |A^{-1}|E|x|\,\|_\infty}{\|x\|_\infty},$$

and their variants. To compute these condition numbers exactly we need to compute A^{-1}, which requires $O(n^3)$ operations, even if we are given a factorization of A. Is it possible to produce reliable estimates of both condition numbers in $O(n^2)$ operations? The answer is yes, but to see why we first need to rewrite the expression for $\operatorname{cond}_{E,f}(A, x)$. Consider the general expression $\| \,|A^{-1}|d\,\|_\infty$, where d is a given nonnegative vector (thus, $d = f + E|x|$ for $\operatorname{cond}_{E,f}(A, x)$); note that the practical error bound (7.27) is of this form. Writing $D = \operatorname{diag}(d)$ and $e = [1, 1, \ldots, 1]^T$, we have

$$\| \,|A^{-1}|d\,\|_\infty = \| \,|A^{-1}|De\,\|_\infty = \| \,|A^{-1}D|e\,\|_\infty = \| \,|A^{-1}D|\,\|_\infty = \|A^{-1}D\|_\infty. \tag{14.1}$$

Hence the problem is equivalent to that of estimating $\|B\|_\infty := \|A^{-1}D\|_\infty$. If we can estimate this quantity then we can estimate any of the condition numbers or perturbation bounds for a linear system. There is an algorithm, described in §14.3, that produces reliable order-of-magnitude estimates of $\|B\|_1$, for an arbitrary B, by computing just a few matrix-vector products Bx and B^Ty for carefully selected vectors x and y and then approximating $\|B\|_1 \approx \|Bx\|_1/\|x\|_1$. If we assume that we have a factorization of A (say, $PA = LU$ or $A = QR$), then we can form the required matrix-vector products for $B = A^{-1}D$ in $O(n^2)$ flops. Since $\|B\|_1 = \|B^T\|_\infty$, it follows that we can use the algorithm to estimate $\| \,|A^{-1}|d\,\|_\infty$ in $O(n^2)$ flops.

Before presenting the l-norm condition estimation algorithm, we describe a more general method that estimates $\|B\|_p$ for any $p \in [1, \infty]$. This more general method is of interest in its own right and provides insight into the special case $p = 1$.

14.2. The p-Norm Power Method

An iterative "power method" for computing $||A||_p$ was proposed by Boyd in 1974 [139, 1974]. When $p = 2$ it reduces to the usual power method applied to A^TA. We use the notation $\text{dual}_p(x)$ to denote any vector y of unit q-norm such that equality holds in the Holder inequality $x^Ty \le \|x\|_p\|y\|_q$ (this normalization is different from the one we used in (6.3), but is more convenient for our present purposes). Throughout this section, $p \ge 1$ and q is defined by $p^{-1} + q^{-1} = 1$.

Algorithm 14.1 (p-norm power method). Given $A \in \mathbb{R}^{m\times n}$ and $x_0 \in \mathbb{R}^n$, this algorithm computes γ and x such that $\gamma \le ||A||_p$ and $||Ax||_p = \gamma||x||_p$.

```
x = x_0 /||x_0||_p
repeat
      y = Ax
      z = A^T dual_p(y)
      if ||z||_q ≤ z^T x
            γ = ||y||_p
            quit
      end
      x = dual_q(z)
end
```

Cost: $4rmn$ flops (for r iterations).

There are several ways to derive Algorithm 14.1. Perhaps the most natural way is to examine the optimality conditions for maximization of

$$F(x) = \frac{\|Ax\|_p}{\|x\|_p}, \quad x \in \mathbb{R}^n.$$

First, we note that the subdifferential (that is, the set of subgradients) of an arbitrary vector norm $\|\cdot\|$ is given by [378, 1987, p. 379]

$$\partial\|x\| = \{\, \lambda : \lambda^Tx = \|x\|,\ \|\lambda\|_D \le 1 \,\}.$$

If $x \neq 0$ then $\lambda^Tx = \|x\| \Rightarrow \|\lambda\|_D \ge 1$, from the Holder inequality, and so, if $x \neq 0$,

$$\begin{aligned}\partial\|x\| &= \{\, \lambda : \lambda^Tx = \|x\|,\ \|\lambda\|_D = 1 \,\} \\ &=: \{\text{dual}(x)\}.\end{aligned}$$

It can also be shown that if A has full rank,

$$\partial\|Ax\| = \{A^T \text{dual}(Ax)\}. \tag{14.2}$$

We assume now that A has full rank, $1 < p < \infty$, and $x \neq 0$. Then it is easy to see that there is a unique vector $\text{dual}_p(x)$, so $\partial\|x\|_p$ has just one element, that is, $\|x\|_p$ is differentiable. Hence we have

$$F(x) = \frac{A^T \text{dual}_p(Ax)}{\|x\|_p} - \frac{\|Ax\|_p \text{dual}_p(x)}{\|x\|_p^2}. \tag{14.3}$$

The first-order Kuhn–Tucker condition for a local maximum of F is therefore

$$A^T \text{dual}_p(Ax) = \frac{\|Ax\|_p}{\|x\|_p} \text{dual}_p(x).$$

Since $\text{dual}_q(\text{dual}_p(x)) = x/\|x\|_p$ if $p \neq 1, \infty$, this equation can be written

$$x = \frac{\|x\|_p^2}{\|Ax\|_p} \text{dual}_q(A^T \text{dual}_p(Ax)). \tag{14.4}$$

The power method is simply functional iteration applied to this transformed set of Kuhn–Tucker equations (the scale factor $\|x\|_p^2/\|Ax\|_p$ is irrelevant since $F(ax) = F(x)$).

For the 1- and ∞–norms, which are not everywhere differentiable, a different derivation can be given. The problem can be phrased as one of maximizing the convex function $F(x) := \|Ax\|_p$ over the convex set $S := \{x : \|x\|_p \leq 1\}$. The convexity of F and S ensures that, for any $u \in S$, at least one vector g exists such that

$$F(v) \geq F(u) + g^T(v - u) \quad \text{for all } v \in S. \tag{14.5}$$

Vectors g satisfying (14.5) are called *subgradients* of F (see, for example, [378, 1987, p. 364]). Inequality (14.5) suggests the strategy of choosing a subgradient g and moving from u to a point $v_* \in S$ that maximizes $g^T(v - u)$, that is, a vector that maximizes $g^T v$. Clearly, $v_* = \text{dual}_q(g)$. Since, from (14.2), g has the form $A^T \text{dual}_p(Ax)$, this completes the derivation of the iteration.

For all values of p the power method has the desirable property of generating an increasing sequence of norm approximations.

Lemma 14.2. *In Algorithm 14.1, the vectors from the kth iteration satisfy*

(i) $z^{k^T} x^k = \|y^k\|_p$, *and*

(ii) $\|y^k\|_p \leq \|z^k\|_q \leq \|y^{k+1}\|_p \leq \|A\|_p$.

The first inequality in (ii) *is strict if convergence is not obtained on the kth iteration.*

Proof. $z^{k^T}x^k = \text{dual}_p(y^k)^T Ax^k = \text{dual}_p(y^k)^T y^k = \|y^k\|_p$. Then

$$\begin{aligned}\|y^k\|_p &= z^{k^T}x^k \\ &\le \|z^k\|_q \|x^k\|_p = \|z^k\|_q = z^{k^T}x^{k+1} = \text{dual}_p(y^k)^T Ax^{k+1} \\ &\le \|\text{dual}_p(y^k)\|_q \|Ax^{k+1}\|_p = \|y^{k+1}\|_p \\ &\le \|A\|_p.\end{aligned}$$

For the last part, note that, in view of (i), the convergence test "$||z^k||_q \le z^{k^T}x^k$" can be written as "$||z^k||_q \le ||y^k||_p$". □

It is clear from Lemma 14.2 (or directly from the Holder inequality) that the convergence test "$||z||_q \le z^T x$" in Algorithm 14.1 is equivalent to "$||z||_q = z^T x$" and, since $||x||_p = 1$, this is equivalent to $x = \text{dual}_q(z)$. Thus, although the convergence test compares two scalars, it is actually testing for equality in the vector equation (14.4).

The convergence properties of Algorithm 14.1 merit a careful description. First, in view of Lemma 14.2, the scalars $\gamma_k = ||y^k||_p$ form an increasing and convergent sequence. This does not necessarily imply that Algorithm 14.1 converges, since the algorithm tests for convergence of the x_k, and these vectors could fail to converge. However, a subsequence of the x^k must converge to a limit, $\bar{x}$ say. Boyd [139, 1974] shows that if $\bar{x}$ is a strong local maximum of F with no zero components, then $x^k \to \bar{x}$ linearly.

If Algorithm 14.1 converges it converges to a stationary point of $F(x)$ when $1 < p < \infty$. Thus, instead of the desired global maximum $||A||_p$, we may obtain only a local maximum or even a saddle point. When $p = 1$ or ∞, if the algorithm converges to a point at which F is not differentiable, that point need not even be a stationary point. On the other hand, for $p = 1$ or ∞ Algorithm 14.1 terminates in at most $n + 1$ iterations (assuming that when dual_p or dual_q is not unique an extreme point of the unit ball is taken), since the algorithm moves between the vertices e_i of the unit ball in the 1-norm, increasing F on each stage ($x = \pm e_i$ for $p = 1$, and $\text{dual}_p(y) = \pm e_i$ for $p = \infty$). An example where n iterations are required for $p = 1$ is given in Problem 14.2.

For two special types of matrix, more can be said about Algorithm 14.1.

(1) If $A = xy^T$ (rank 1), the algorithm converges on the second step with $\gamma = ||A||_p = ||x||_p||y||_q$, whatever x_0.

(2) Boyd [139, 1974] shows that if A has nonnegative elements, A^TA is irreducible, $1 < p < \infty$, and x_0 has positive elements, then the x^k converge and $\gamma_k \to ||A||_p$.

For values of p strictly between 1 and ∞ the convergence behaviour of Algorithm 14.1 is typical of a linearly convergent method: exact convergence is not usually obtained in a finite number of steps and arbitrarily many steps can

be required for convergence, as is well-known for the 2-norm power method. Fortunately, there is a method for choosing a good starting vector that can be combined with Algorithm 14.1 to produce a reliable norm estimator; see the Notes and References and Problem 14.1.

We now turn our attention to the extreme values of p: 1 and ∞.

14.3. LAPACK 1-Norm Estimator

Algorithm 14.1 has two remarkable properties when p = 1: it almost always converges within four iterations (when $x_0 = [1, 1, \ldots, 1]^T$, say) and it frequently yields $\|A\|_1$ exactly. This rapid, finite termination is also obtained for $p = \infty$, and is related to the fact that Algorithm 14.1 moves among the finite set of extreme points of the unit ball. Numerical experiments suggest that the accuracy is about the same for both norms but that slightly more iterations are required on average for $p = \infty$. Hence we will confine our attention to the 1-norm.

The 1-norm version of Algorithm 14.1 was derived independently of the general algorithm by Hager [492, 1984] and can be expressed as follows. The notation $\xi = \text{sign}(y)$ means that $\xi_i = 1$ or -1 according as $y_i \geq 0$ or $y_i < 0$. We now specialize to square matrices.

Algorithm 14.3 (1-norm power method). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and x such that $\gamma \leq \|A\|_1$ and $\|Ax\|_1 = \gamma \|x\|_1$.

```
x = n^{-1} e
repeat
    y = Ax
    ξ = sign(y)
    z = A^T ξ
    if ||z||_∞ ≤ z^T x
        γ = ||y||_1
        quit
    end
    x = e_j, where |z_j| = ||z||_∞ (smallest such j)
end
```

Numerical experiments show that the estimates produced by Algorithm 14.3 are frequently exact ($\gamma = \|A\|_1$), usually "acceptable" ($\gamma \geq \|A\|_1/10$), and sometimes poor ($\gamma < \|A\|_1/10$).

An important question for any norm or condition estimator is whether there exists a "counterexample"—a parametrized matrix for which the quotient "estimate divided by true norm" can be made arbitrarily small (or large, depending on whether the estimator produces a lower bound or an upper

bound) by varying a parameter. A general class of counterexample for Algorithm 14.3 is given by the matrices

$$A = I + \theta C,$$

where $Ce = C^T e = 0$ (there are many possible choices for C). For any such matrix, Algorithm 14.3 computes $y = n^{-1}e$, $\xi = e$, $z = e$, and hence the algorithm terminates at the end of the first iteration with

$$\frac{\gamma}{\|A\|_1} = \frac{1}{\|I + \theta C\|_1} \sim \theta^{-1} \quad \text{as } \theta \to \infty.$$

The problem is that the algorithm stops at a local maximum that can differ from the global one by an arbitrarily large factor.

A more reliable and more robust algorithm is produced by the following modifications of Higham [537, 1988].

Definition of estimate. To overcome most of the poor estimates, γ is redefined as

$$\gamma = \max\left\{ \|y\|_1, \frac{\|c\|_1}{\|b\|_1} \right\},$$

where

$$c = Ab, \quad b_i = (-1)^{i+1}\left(1 + \frac{i-1}{n-1}\right).$$

The vector b is considered likely to "pick out" any large elements of A in those cases where such elements fail to propagate through to y.

Convergence test. The algorithm is limited to a minimum of two and a maximum of five iterations. Further, convergence is declared after computing ξ if the new ξ is the same as the previous one; this event signals that convergence will be obtained on the current iteration and that the next (and final) multiplication $A^T\xi$ is unnecessary. Convergence is also declared if the new $\|y\|_1$ is no larger than the previous one. This nonincrease of the norm can happen only in finite precision arithmetic and signals the possibility of a vertex e_j being revisited—the onset of "cycling."

The improved algorithm is as follows. This algorithm is the basis of all the condition number estimation in LAPACK.

Algorithm 14.4 (LAPACK norm estimator). Given $A \in \mathbb{R}^{n\times n}$ this algorithm computes γ and $v = Aw$ such that $\gamma \le \|A\|_1$ with $\|v\|_1/\|w\|_1 = \gamma$ (w is not returned).

$v = A(n^{-1}e)$
if $n = 1$, quit with $\gamma = |v_1|$, end
$\gamma = \|v\|_1$
$\xi = \operatorname{sign}(v)$

$x = A^T\xi$
$k = 2$
repeat
  $j = \min\{i: |x_i| = \|x\|_\infty\}$
  $v = Ae_j$
  $\overline{\gamma} = \gamma$
  $\gamma = \|v\|_1$
  if $\mathrm{sign}(v) = \xi$ or $\gamma \le \overline{\gamma}$, goto (*), end
  $\xi = \mathrm{sign}(v)$
  $x = A^T\xi$
  $k = k+1$
until $(\|x\|_\infty = x_j$ or $k > 5)$
(*) $x_i = (-1)^{i+1}\left(1 + \frac{i-1}{n-1}\right), \quad i = 1{:}n$
$x = Ax$
if $2\|x\|_1/(3n) > \gamma$ then
  $v = x$
  $\gamma = 2\|x\|_1/(3n)$
end

Algorithm 14.4 can still be "defeated": it returns an estimate 1 for matrices $A(\theta)$ of the form

$$A(\theta) = I + \theta P, \text{ where } P = P^T,\ Pe = 0,\ Pe_1 = 0,\ Pb = 0. \qquad (14.6)$$

(P can be constructed as $I - Q$ where Q is the orthogonal projection onto span{e, e_1, b}.) Indeed, the existence of counterexamples is intuitively obvious since Algorithm 14.4 samples the behaviour of A on less than n vectors in $\mathbb{R}^n$. Numerical counterexamples (not parametrized) can be constructed automatically by direct search, as described in §24.3.1. Despite these weaknesses, practical experience with Algorithm 14.4 shows that it is very rare for the estimate to be more than three times smaller than the actual norm, independent of the dimension n. Therefore Algorithm 14.4 is, in practice, a very *reliable* norm estimator. The number of matrix-vector products required is at least 4 and at most 11, and averages between 4 and 5.

There is an analogue of Algorithm 14.3 for complex matrices, in which ξ_i is defined as $y_i/|y_i|$ if $y_i \neq 0$ and 1 otherwise. In the corresponding version of Algorithm 14.4 the test for repeated ξ vectors is removed, because ξ now has noninteger, complex components and so is unlikely to repeat.

It is interesting to look at a performance profile of Algorithm 14.4. A performance profile is a plot of some measure of the performance of an algorithm versus a problem parameter. In this case, the natural measure of performance is the underestimation ratio, $\gamma/\|A\|_1$. Figure 14.1 shows the performance profile for a 5 × 5 matrix $A(\theta)$ of the form (14.6), with P constructed as

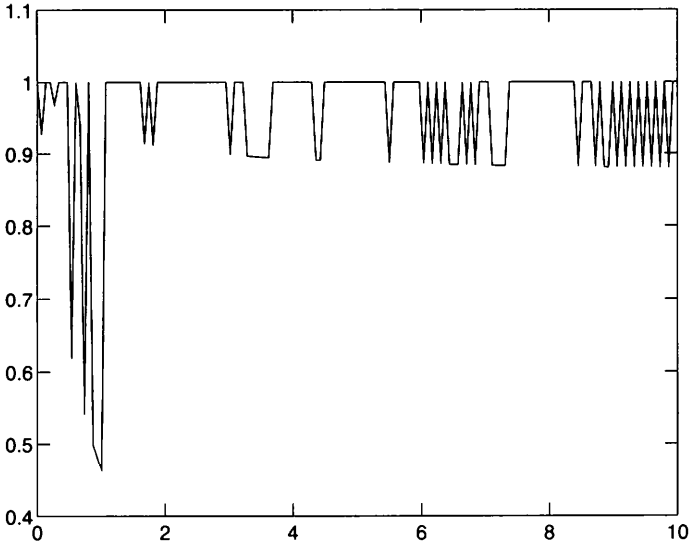

Figure 14.1. *Underestimation ratio for Algorithm* 14.4 *for* 5 × 5 *matrix A(O) of* (14.6) *with 150 equally spaced values of* $\theta \in [0,10]$.

described above (because of rounding errors in constructing $A(\theta)$ and within the algorithm, the computed norm estimates differ from those that would be produced inexact arithmetic). The jagged nature of the performance curve is typical for algorithms that contain logical tests and branches. Small changes in the parameter θ, which themselves result in different rounding errors, can cause the algorithm to visit different vertices in this example.

14.4. Other Condition Estimators

The first condition estimator to be widely used is the one employed in LINPACK. It was developed by Cline, Moler, Stewart, and Wilkinson [216, 1979]. The idea behind this condition estimator originates with Gragg and Stewart [476, 1976], who were interested in computing an approximate null vector rather than estimating the condition number itself.

We will describe the algorithm as it applies to a triangular matrix $T \in \mathbb{R}^{n\times n}$. There are three steps:

1. Choose a vector d such that $||y||$ is as large as possible relative to $||d||$, where $T^T y = d$.

2. Solve $Tx = y$.

3. Estimate $\|T^{-1}\| \approx \|x\|/\|y\|$ $(\leq \|T^{-1}\|)$.

In LINPACK the norm is the l-norm, but the algorithm can also be used for the 2-norm or the cm-norm. The motivation for step 2 is based on a singular value decomposition analysis. Roughly, if $||y||/||d||$ $(\approx \|T^{-T}\|)$ is large then $\|x\|/\|y\|$ $(\approx \|T^{-1}\|)$ will almost certainly be at least as large, and it could be a much better estimate. Notice that $T^T Tx = d$, so the algorithm is related to the power method on the matrix $(T^T T)^{-1}$ with the specially chosen starting vector d.

To examine step 1 more closely, suppose that $T = U^T$ is lower triangular and note that the equation $Uy = d$ can be solved by the following column-oriented (saxpy) form of substitution:

$p(1:n) = 0$
for $j = n:-1:1$
  $y_j = (d_j - p_j)/u_{jj}$
(*) $p(1:j-1) = p(1:j-1) + U(1:j-1, j)y(j)$
end

The idea is to choose the elements of the right-hand side vector d adaptively as the solution proceeds, with $d_j = \pm 1$. At the jth stage of the algorithm $d \ldots, d_{j+1}$ have been chosen and $y_n, \ldots, y_{j+1}$ are known. The next element $d_j \in \{+1, -1\}$ is chosen so as to maximize a weighted sum of $d_j - p_j$ and the partial sums $p_1, \ldots, p_j$, *which* would be computed during the next execution of statement (*) above. Hence the algorithm looks ahead, trying to gauge the effect of the choice of d_j on future solution components. This heuristic algorithm for choosing d is expressed in detail as follows.

Algorithm 14.5 (LINPACK condition estimator). Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$ and a set of nonnegative weights $\{w_i\}$, this algorithm computes a vector y such that $Uy = d$, where the elements $d_j = \pm 1$ are chosen to make $||y||$ large.

$p(1:n) = 0$
for $j = n:-1:1$
  $y_j^+ = (1 - p_j)/u_{jj}$
  $y_j^- = (-1 - p_j)/u_{jj}$
  $p^+(1:j-1) = p(1:j-1) + U(1:j-1, j)y^+(j)$
  $p^-(1:j-1) = p(1:j-1) + U(1:j-1, j)y^-(j)$
  if $w_j|1 - p_j| + \sum_{i=1}^{j-1} w_i|p_i^+| \geq w_j|1 + p_j| + \sum_{i=1}^{j-1} w_i|p_i^-|$
    $y_j = y_j^+$
    $p(1:j-1) = p^+(1:j-1)$
  else
    $y_j = y_j^-$

```
      p(1:j-1) = p-(1:j-1)
    end
end
```

cost: $4n^2$ flops.

LINPACK takes the weights $w_j \equiv 1$, though another possible (but more expensive) choice would be $w_j = 1/|u_{jj}|$, which corresponds to how p_j is weighted in the expression $y_j = (d_j - p_j)/u_{jj}$.

To estimate $||A^{-1}||$ for a full matrix A, the LINPACK estimator makes use of an LU factorization of A. Given $PA = LU$, the equations solved are $U^T z = d$, $L^T y = z$, and $AX = P^T y$, where for the first system d is constructed by the analogue of Algorithm 14.5 for lower triangular matrices; the estimate is $||x||_1/||y||_1 \approx ||A^{-1}||_1$. Since d is chosen without reference to L, there is an underlying assumption that any ill condition in A is reflected in U. This assumption may not be true; see Problem 14.3.

In contrast to the LAPACK norm estimator, the LINPACK estimator requires explicit access to the elements of the matrix. Hence the estimator cannot be used to estimate componentwise condition numbers. Furthermore, separate code has to be written for each different type of matrix and factorization. Consequently, while LAPACK has just a single norm estimation routine, which is called by many other routines, LINPACK has multiple versions of its algorithm, each tailored to the specific matrix or factorization.

Several years after the LINPACK condition estimator was developed, several parametrized counterexamples were found by Cline and Rew [217, 1983]. Numerical counterexamples can also be constructed by direct search, as shown in §24.3.1. Despite the existence of these counterexamples the LINPACK estimator has been widely used and is regarded as being almost certain to produce an estimate correct to within a factor 10 in practice.

A 2-norm condition estimator was developed by Cline, Corm, and Van Loan [218, 1982, Algorithm 1]; see also Van Loan [1043, 1987] for another explanation. The algorithm builds on the ideas underlying the LINPACK estimator by using "look-behind" as well as look-ahead. It estimates $\sigma_{\min}(R) = \|R^{-1}\|_2^{-1}$ or $\sigma_{\max}(R) = ||R||_2$ for a triangular matrix R, where $\sigma_{\min}$ and $\sigma_{\max}$ denote the smallest and largest singular values, respectively. Full matrices can be treated if a factorization $A = QR$ is available (Q orthogonal, R upper triangular), since R and A have the same singular values. The estimator performs extremely well in numerical tests, often producing an estimate that has some correct digits [218, 1982], [534, 1987]. No counterexamples to the estimator were known until Bischof [103, 1990] obtained counterexamples as a by-product of the analysis of a different but related method, mentioned at the end of this section.

All the methods described so far have the property that when applied repeatedly to a given matrix they always produce the same estimate. Another

approach is to introduce some randomness, so that the output of the method depends on the particular random numbers chosen. A natural idea along these lines is to apply the power method to the matrix $(AA^T)^{-1}$ with a randomly chosen starting vector. If a factorization of A is available, the power method vectors can be computed inexpensively by solving linear systems with A and A^T. Analysis based on the singular value decomposition suggests that there is a high probability that a good estimate of $\|A^{-1}\|_2$ will be obtained. This notion is made precise by Dixon [306, 1983], who proves the following result.

Theorem 14.6 (Dixon). *Let $A \in \mathbb{R}^{n\times n}$ be nonsingular and let $\theta > 1$ be a constant. If $x \in \mathbb{R}^n$ is a random vector from the uniform distribution on the unit sphere $S_n = \{ y \in \mathbb{R}^n : y^T y = 1 \}$, then the inequality*

$$\left(x^T (AA^T)^{-k} x\right)^{1/2k} \le \|A^{-1}\|_2 \le \theta \left(x^T (AA^T)^{-k} x\right)^{1/2k} \tag{14.7}$$

holds with probability at least $1 - 0.8\theta^{-k/2} n^{1/2}$ $(k \ge 1)$. □

Note that the left-hand inequality in (14.7) always holds; it is only the right-hand inequality that is in question.

For $k = 1$, (14.7) can be written as

$$\|A^{-1}x\|_2 \le \|A^{-1}\|_2 \le \theta \|A^{-1}x\|_2,$$

which suggests the simple estimate $\|A^{-1}\|_2 \approx \|A^{-1}x\|_2$, where x is chosen randomly from the uniform distribution on S_n. Such vectors x can be generated from the formula

$$x_i = z_i / \|z\|_2,$$

where $z_1, \ldots, z_n$ are independent random variables from the normal $N(0, 1)$ distribution [668, 1981, p. 130]. If, for example, $n = 100$ and θ has the rather large value 6400 then inequality (14.7) holds with probability at least 0.9.

In order to take a smaller constant θ, for fixed n and a desired probability, we can use larger values of k. If $k = 2j$ is even then we can simplify (14.7), obtaining

$$\|(AA^T)^{-j} x\|_2^{1/2j} \le \|A^{-1}\|_2 \le \theta \|(AA^T)^{-j} x\|_2^{1/2j}, \tag{14.8}$$

and the minimum probability stated by the theorem is $1 - 0.8\theta^{-j} n^{1/2}$. Taking $j = 3$, for the same value $n = 100$ as before, we find that (14.8) holds with probability at least 0.9 for the considerably smaller value $\theta = 4.31$.

Probabilistic condition estimation has not yet been adopted in any major software packages, perhaps because the other techniques work so well. For more on the probabilistic power method approach see Dixon [306, 1983], Higham [534, 1987], and Kuczynski and Wozniakowski [676, 1992] (who also analyse the more powerful Lanczos method with a random starting vector). For a probabilistic condition estimation method of very general applicability

see Kenney and Laub [652, 1994] and Gudmundsson, Kenney, and Laub [485, 1995].

The condition estimators described above assume that a single estimate is required for a matrix given in its entirety. Condition estimators have also been developed for more specialized situations. Bischof [103, 1990] develops a method for estimating the smallest singular value of a triangular matrix which processes the matrix a row or a column at a time. This "incremental condition estimation" method can be used to monitor the condition of a triangular matrix as it is generated, and so is useful in the context of matrix factorization such as the QR factorization with column pivoting. The estimator is generalized to sparse matrices by Bischof, Lewis, and Pierce [104, 1990]. Barlow and Vemulapati [67, 1992] develop a l-norm incremental condition estimator with look-ahead for sparse matrices.

Condition estimates are also required in applications where a matrix factorization is repeatedly updated as a matrix undergoes low rank changes. Algorithms designed for a recursive least squares problem and employing the Lanczos method are described by Ferng, Golub, and Plemmons [372, 1991]. Pierce and Plemmons [831, 1992] describe an algorithm for use with the Cholesky factorization as the factorization is updated, while Shroff and Bischof [918, 1992] treat the QR factorization.

14.5. Condition Numbers of Tridiagonal Matrices

For a bidiagonal matrix B, $|B^{-1}| = M(B)^{-1}$ (see §8.3), so the condition numbers $\kappa_{E,f}$ and $\mathrm{cond}_{E,f}$ can be computed exactly with an order of magnitude less work than is required to compute B^{-1} explicitly. This property holds more generally for several types of tridiagonal matrix, as a consequence of the following result. Recall that the LU factors of a tridiagonal matrix are bidiagonal and may be computed using the formulae (9.16).

Theorem 14.7. *If the nonsingular tridiagonal matrix $A \in \mathbb{R}^{n\times n}$ has the LU factorization $A = LU$ and $|L||U| = |A|$, then $|U^{-1}||L^{-1}| = |A^{-1}|$.*

Proof. Using the notation of (9.15), $|L||U| = |A| = |LU|$ if and only if, for all 2,

$$|l_i e_{i-1} + u_i| = |l_i||e_{i-1}| + |u_i|,$$

that is, if

$$\operatorname{sign}\left(\frac{l_i e_{i-1}}{u_i}\right) = 1. \tag{14.9}$$

Using the formulae

$$\left(U^{-1}\right)_{ij} = \frac{1}{u_j}\prod_{p=i}^{j-1}\left(\frac{-e_p}{u_p}\right), \qquad j \geq i,$$

$$\left(L^{-1}\right)_{ij} = \prod_{p=j}^{i-1}(-l_{p+1}), \qquad i \geq j,$$

we have

$$\begin{aligned}
\left(U^{-1}L^{-1}\right)_{ij} &= \sum_{k=\max(i,j)}^{n} \left(U^{-1}\right)_{ik}\left(L^{-1}\right)_{kj} \\
&= \sum_{k=\max(i,j)}^{n} \frac{1}{u_k} \prod_{p=i}^{k-1}\left(\frac{-e_p}{u_p}\right) \prod_{p=j}^{k-1}(-l_{p+1}) \\
&= \prod_{p=i}^{\max(i,j)-1}\left(\frac{-e_p}{u_p}\right) \cdot \prod_{p=j}^{\max(i,j)-1}(-l_{p+1}) \\
&\quad \times \sum_{k=\max(i,j)}^{n} \frac{1}{u_k} \prod_{p=\max(i,j)}^{k-1}\left(\frac{e_p l_{p+1}}{u_p}\right) \\
&= \prod_{p=i}^{\max(i,j)-1}\left(\frac{-e_p}{u_p}\right) \cdot \prod_{p=j}^{\max(i,j)-1}(-l_{p+1}) \\
&\quad \times \frac{1}{u_{\max(i,j)}} \cdot \sum_{k=\max(i,j)}^{n} \prod_{p=\max(i,j)}^{k-1}\left(\frac{e_p l_{p+1}}{u_{p+1}}\right).
\end{aligned}$$

Thus, in view of (14.9), it is clear that $|U^{-1}L^{-1}|_{ij} = (|U^{-1}||L^{-1}|)_{ij}$, as required. □

Since L and U are bidiagonal, $|U^{-1}| = M(U)^{-1}$ and $|L^{-1}| = M(L)^{-1}$. Hence, if $|A| = |L||U|$, then, from Theorem 14.7,

$$|A^{-1}|y = |U^{-1}||L^{-1}|y = M(U)^{-1}M(L)^{-1}y. \tag{14.10}$$

It follows that we can compute any of the condition numbers or forward error bounds of interest *exactly* by solving two bidiagonal systems. The cost is $O(n)$ flops, as opposed to the $O(n^2)$ flops needed to compute the inverse of a tridiagonal matrix.

When does the condition $|A| = |L||U|$ hold? Theorem 9.11 shows that it holds if the tridiagonal matrix A is symmetric positive definite, totally positive, or an M-matrix. So for these types of matrix we have a very satisfactory way to compute the condition number.

If A is tridiagonal and diagonally dominant by rows, then we can compute in $O(n)$ flops an upper bound for the condition number that is not more than a factor $2n - 1$ too large.

Theorem 14.8. *Suppose the nonsingular, row diagonally dominant tridiagonal matrix $A \in \mathbb{R}^{n\times n}$ has the LU factorization $A = LU$. Then, if $y \geq 0$,*

$$\| |U^{-1}||L^{-1}|y \|_\infty \leq (2n-1)\| |A^{-1}|y \|_\infty.$$

Proof. We have $L^{-1} = UA^{-1}$, so

$$|U^{-1}||L^{-1}|y \leq |U^{-1}||U||A^{-1}|y = |V^{-1}||V||A^{-1}|y,$$

where the bidiagonal matrix $V = \mathrm{diag}(u_{ii})^{-1} U$ has $v_{ii} \equiv 1$ and $|v_{i,i+1}| = |e_i/u_i| \leq 1$ (see the proof of Theorem 9.12). Thus

$$|U^{-1}||L^{-1}|y \leq \begin{bmatrix} 1 & 1 & \dots & 1 \\ & 1 & \dots & 1 \\ & & \ddots & \vdots \\ & & & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & & \\ & 1 & \ddots & \\ & & \ddots & 1 \\ & & & 1 \end{bmatrix} |A^{-1}|y,$$

and the result follows on taking norms. □

In fact, it is possible to compute $|A^{-1}|y$ exactly in $O(n)$ operations for any tridiagonal matrix. This is a consequence of the special form of the inverse of a tridiagonal matrix.

Theorem 14.9 (Ikebe). *Let $A \in \mathbb{R}^{n\times n}$ be tridiagonal and irreducible (that is, $a_{i+1,i}$ and $a_{i,i+1}$ are nonzero for all i). Then there are vectors x, y, p, and q such that*

$$(A^{-1})_{ij} = \begin{cases} x_i y_j, & i \leq j, \\ p_i q_j, & i \geq j. \end{cases} \qquad \square$$

This result says that the inverse of an irreducible tridiagonal matrix is the upper triangular part of a rank-1 matrix joined along the diagonal to the lower triangular part of another rank-1 matrix. If A is reducible then it has the block form $\begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}$ (or its transpose), and this blocking can be applied recursively until the diagonal blocks are all irreducible, at which point the theorem can be applied to the diagonal blocks.

The vectors x, y, p, and q in Theorem 14.9 can all be computed in $O(n)$ flops, and this enables the condition numbers and forward error bounds to be computed also in $O(n)$ flops (see Problem 14.5). Unfortunately, the vectors x, y, p, and q can have a huge dynamic range, causing the computation to break down because of overflow or underflow. For example, for the diagonally dominant tridiagonal matrix with $a_{ii} \equiv 4$, $a_{i+1,i} = a_{i,i+1} \equiv 1$, we have ($x_1 = 1$) $|x_n| \approx \theta^{n-1}$, $|y1| \approx \theta^{-1}$, and $|y_n| \approx \theta^{-n}$, where $\theta = 2 + \sqrt{3} \approx 3.73$. These numerical problems can be overcome, but only at a nontrivial increase in cost. Therefore, we do not recommend the use of Theorem 14.9 for computing condition numbers. For a general tridiagonal matrix it is probably better to *estimate* the condition number using Algorithm 14.4.

14.6. Notes and References

The clever trick (14.1) for converting the norm $\| |A^{-1}|d \|_\infty$ into the norm of a matrix with which products are easily formed is due to Arioli, Demmel, and Duff [24, 1989].

The p-norm power method was first derived and analysed by Boyd [139, 1974] and was later investigated by Tao [993, 1984]. Tao applies the method to an arbitrary mixed subordinate norm $||A||_{a,\beta}$ (see (6.6)), while Boyd takes the a and β-norms to be p-norms (possibly different). Algorithm 14.1 can be converted to estimate $||A||_{a,\beta}$ by making straightforward modifications to the norm-dependent terms. An algorithm that estimates $||A||_p$ using the power method with a specially chosen starting vector is developed by Higham [551, 1992]; the method for obtaining the starting vector is outlined in Problem 14.1. The estimate produced by this algorithm is always within a factor $n^{1-1/p}$ of $||A||_p$ and the cost is about $70n^2$ flops. A MATLAB M-file pnorm implementing this method is part of the Test Matrix Toolbox (see Appendix E).

The finite convergence of the power method for $p = 1$ and p = 00 holds more generally: if the power method is applied to the norm $||\cdot||_{a,\beta}$ and one of the a and β norms is polyhedral (that is, its unit ball has a finite number of extreme points), then the iteration converges in a finite number of steps. Moreover, under a reasonable assumption, this number of steps can be bounded in terms of the number of extreme points of the unit balls in the a-norm and the dual of the β-norm. See Bartels [75, 1991] and Tao [993, 1984] for further details.

Hager [492, 1984] gave a derivation of the 1-norm estimator based on subgradients and used the method to estimate $\kappa_1(A)$. That the method is of wide applicability y because it accesses A only through matrix-vector products was recognized by Higham, who developed Algorithm 14.4 and its complex analogue and wrote Fortran 77 implementations, which use a reverse communication interface [537, 1988], [543, 1990]. These codes are used in LAPACK, the NAG library, and various other program libraries. A version of Algorithm 14.4 dedicated to estimating $\kappa_1(A)$ is supplied with MATLAB as M-file condest. Algorithm 14.4 is also implemented in ROM on the Hewlett-Packard HP 48G and HP 48GX calculators (along with several other LAPACK routines), in a form that estimates $\kappa_1(A)$. The Hewlett-Packard implementation is instructive because it shows that condition estimation can be efficient even for small dimensions: on a standard HP 48G, inverting A and estimating its condition number (without being given a factorization of A in either case) both take about 5 seconds for $n = 10$, while for $n = 20$ inversion takes 30 seconds and condition estimation only 20 seconds.

Moler [769, 1978] describes an early version of the LINPACK condition estimator and raises the question of the existence of counterexamples. An early version without look-ahead was incorporated in the Fortran code decomp

in the book of Forsythe, Malcolm, and Moler [395, 1977].

Matrices for which condition estimators perform poorly can be very hard to find theoretically or with random testing, but for all the estimators described in this chapter they can be found quite easily by applying direct search optimization to the under- or overestimation ratio; see §24.3.1.

Both LINPACK and LAPACK return estimates of the reciprocal of the condition number, in a variable rcond ≤ 1. Overflow for a very ill conditioned matrix is thereby avoided, and rcond is simply set to zero when singularity is detected. MATLAB has a built-in function rcond that implements the LINPACK condition estimation algorithm.

A simple modification to the LINPACK estimator that can produce a larger estimate is suggested by O'Leary [804, 1980]. For sparse matrices, Grimes and Lewis [483, 1981] suggest a way to reduce the cost of the scaling strategy used in LINPACK to avoid overflow in the condition estimation. Zlatev, Wasniewski, and Schaumburg [1134, 1986] describe their experience in implementing the LINPACK condition estimation algorithm in a software package for sparse matrices.

Stewart [946, 1980] describes an efficient way to generate random matrices of a given condition number and singular value distribution (see §26.3) and tests the LINPACK estimator on such random matrices.

Condition estimators specialized to the (generalized) Sylvester equation have been developed by Byers [172, 1984], Kågström and Westin [624, 1989], and Kågström and Poromaa [621, 1992].

A survey of condition estimators up to 1987, which includes counterexamples and the results of extensive numerical tests, is given by Higham [534, 1987].

Theorems 14.7 and 14.8 are from Higham [541, 1990]. That $\|A^{-1}\|_\infty$ can be computed in $O(n)$ flops for symmetric positive definite tridiagonal A was first, shown in Higham [531, 1986].

Theorem 14.9 has a long history, having been discovered independently in various forms by different authors. The earliest reference we know for the result as stated is Ikebe [600, 1979], where a more general result for Hessenberg matrices is proved. A version of Theorem 14.9 for symmetric tridiagonal matrices was proved by Bukhberger and Emel'yanenko [157, 1973]. The culmination of the many papers on inverses of tridiagonal and Hessenberg matrices is a result of Cao and Stewart on the form of the inverse of a block matrix (A_{ij}) with $A_{ij} = 0$ for $i > j + s$ [184, 1986]; despite the generality of this result, the proof is short and elegant. *Any* banded matrix has an inverse with a special "low rank" structure; the earliest reference on the inverse of a general band matrix is Asplund [32, 1959]. For a recent survey on the inverses of symmetric tridiagonal and block tridiagonal matrices see Meurant [751, 1992].

For symmetric positive definite tridiagonal A the standard way to solve

$AX = b$ is by using a Cholesky or LDL^T factorization, rather than an LU factorization. The LINPACK routine **SPTSL** uses a nonstandard "LUB" factorization resulting from the BABE ("burn at both ends") algorithm, which eliminates from the middle of the matrix to the top and bottom simultaneously (see the LINPACK Users' Guide [307, 1979, Chap. 7] and Higham [531, 1986]). The results of §14.5 are applicable to all these factorization, with minor modifications.

14.6.1. LAPACK

Algorithm 14.4 is implemented in routine `xLACON`, which has a reverse communication interface. The LAPACK routines `xPTCON` and `xPTRFS` for symmetric positive definite tridiagonal matrices compute condition numbers using (14.10); the LAPACK routines `xGTCON` and `xGTRFS` for general tridiagonal matrices use Algorithm 14.4. LINPACK'S tridiagonal matrix routines do not incorporate condition estimation.

Problems

14.1. (Higham [551, 1992]) The purpose of this problem is to develop a noniterative method for choosing a starting vector for Algorithm 14.1. The idea is to choose the components of x in the order $x_1, x_2, \ldots, x_n$ in an attempt to maximize $||Ax||_p/||x||_p$. Suppose $x_1, \ldots, x_{k-1}$ satisfying $||x(1:k\text{-}1)||_p = 1$ have been determined and let $\gamma_{k-1} = ||A(:, 1:k\text{-}1)x(1:k\text{-}1)||_p$. We now try to choose x_k, and at the same time revise $x(1:k-1)$, to give the next partial product a larger norm. Defining

$$g(\lambda, \mu) = \|\lambda A(:, 1:k-1)x(1:k-1) + \mu A(:, k)\|_p$$

we set

$$x_k = \mu^*, \qquad x(1:k-1) \leftarrow \lambda^* x(1:k-1),$$

where

$$g(\lambda^*, \mu^*) = \max\left\{ g(\lambda, \mu) : \left\| \begin{bmatrix} \lambda \\ \mu \end{bmatrix} \right\|_p = 1 \right\}.$$

Then $||x(1:k)||_p = 1$ and

$$\gamma_k = \|A(:, 1:k)x(1:k)\|_p \geq \gamma_{k-1}.$$

Develop this outline into a practical algorithm. What can you prove about the quality of the estimate $||Ax||_p/||x||_p$ that it produces?

14.2. (Higham [543, 1990]) Let the $n \times n$ symmetric tridiagonal matrix $T_n(a) = (t_{ij})$ be defined by

$$t_{ii} = \begin{cases} 2, & i = 1, \\ i, & 2 \le i \le n-1, \\ -t_{n,n-1} + \alpha, & i = n, \end{cases}$$

$$t_{i,i+1} = \begin{cases} -((i+1)/2 - \alpha) & \text{if } i \text{ is odd}, \\ -i/2 & \text{if } i \text{ is even}. \end{cases}$$

For example, $T_6(a)$ is given by

$$\begin{bmatrix} 2 & -(1-\alpha) & & & & \\ -(1-\alpha) & 2 & -1 & & & \\ & -1 & 3 & -(2-\alpha) & & \\ & & -(2-\alpha) & 4 & -2 & \\ & & & -2 & 5 & -(3-\alpha) \\ & & & & -(3-\alpha) & 3 \end{bmatrix}.$$

Note that, for all a, $||T_n(a)e_{n-1}||_1 = ||T_n(a)||_1$. Show that if Algorithm 14.3 is applied to $T_n(a)$ with $0 \le a < 1$ then $x = e_{i-1}$ on the ith iteration, for $i = 2, \ldots, n$, with convergence on the nth iteration. Algorithm 14.4, however, terminates after five iterations with $y^5 = T_n(a)e_4$, and

$$\frac{\|y^5\|_1}{\|T_n(\alpha)\|_1} = \frac{8-\alpha}{2n-2-\alpha} \to 0 \quad \text{as } n \to \infty.$$

Show that the extra estimate saves the day, so that Algorithm 14.4 returns a final estimate that is within a factor 3 of the true norm, for any $a < 1$.

14.3. Let $PA = LU$ be an LU factorization with partial pivoting of A 1 $\mathbb{R}^{n\times n}$. Show that

$$\frac{\|A^{-1}\|_\infty}{2^{n-1}} \le \|U^{-1}\|_\infty \le n\|A^{-1}\|_\infty.$$

14.4. (Higham [537, 1988]) Investigate the behaviour of Algorithms 14.3 and **14.4** for the Pei matrix, $A = aI + ee^T$ ($a \ge 0$), and for the upper bidiagonal matrix with 1s on the diagonal and the first superdiagonal.

14.5. (Ikebe [600, 1979], Higham [531, 1986]) Let $A \in \mathbb{R}^{n\times n}$ 1 be nonsingular, tridiagonal, and irreducible. By equating the last columns in $AA^{-1} = I$ and the first rows in $A^{-1}A = I$, show how to compute the vectors x and y in Theorem 14.9 in $O(n)$ flops. Hence obtain an $O(n)$ flops algorithm for computing $\| |A^{-1}|d \|_\infty$, where $d \ge 0$.

14.6. The representation of Theorem 14.9 for the inverse of nonsingular, tridiagonal, and irreducible $A \in \mathbb{R}^{n\times n}$ involves 472 parameters, yet A depends only on $3n - 2$ parameters. Obtain an alternative representation that involves only $3n - 2$ parameters. (Hint: symmetrize the matrix.)

14.7. (RESEARCH PROBLEM) (Demmel [286, 1992]) Show that estimating $\|A^{-1}\|$ to within a factor depending only on the dimension of A is at least as expensive as computing A^{-1}.

14.8. (RESEARCH PROBLEM) Let $A \in \mathbb{R}^{n\times n}$ be diagonally dominant by rows, let $A = LU$ be an LU factorization, and let $y \geq 0$. What is the maximum size of $\| |U^{-1}||L^{-1}|y \|_\infty / \|A^{-1}y\|_\infty$? This is an open problem raised in [541, 1990]. In a small number of numerical experiments with full random matrices the ratio has been found to be less than 2 [541, 1990], [790, 1986].

Chapter 15
The Sylvester Equation

We must commence, not with a *square,*
but with an oblong arrangement of terms consisting, suppose,
of m lines and n columns.
This will not in itself represent a determinant,
but is, as it were, a Matrix out of which we may form
various systems of determinants by fixing upon a number p,
and selecting at will p lines and p columns,
the squares corresponding to which may be termed
*determinants of the p*th *order.*

— J. J. SYLVESTER, *Additions to the Articles, "On a New Class of Theorems, " and "On Pasta/'s Theorem" (1850)*

I have in previous papers defined a "Matrix" as a rectangular array of terms,
out of which different systems of determinants may be engendered,
as from the womb of a common parent;
these cognate determinants being
by no means isolated in their relations to one another,
but subject to certain simple laws of
mutual dependence and simultaneous deperition.

— J. J. SYLVESTER, *On the Relation Between the Minor Determinants of Linear/y Equivalent Quadratic Functions (1851)*